A Real-Time Physics Framework, with advanced Cutting, Tearing and Piercing interactions for mobile XR applications

Michail Tamiolakis

Thesis submitted in partial fulfillment of the requirements for the

Masters' of Science degree in Computer Science and Engineering

University of Crete
School of Sciences and Engineering
Computer Science Department
Voutes University Campus, 700 13 Heraklion, Crete, Greece

Thesis Advisor: Prof. *George Papagiannakis*

This work has been performed at the University of Crete, School of Sciences and Engineering, Computer Science Department.

The work has been supported by the Foundation for Research and Technology - Hellas (FORTH), Institute of Computer Science (ICS).

University of Crete
Computer Science Department

A Real-Time Physics Framework, with advanced Cutting, Tearing and Piercing interactions for mobile XR applications

Thesis submitted by
Michail Tamiolakis
in partial fulfillment of the requirements for the
Masters' of Science degree in Computer Science

THESIS APPROVAL

Author: Michail Tamiolakis

Committee approvals:

George Papagiannakis
Professor, Thesis Supervisor

Antonios Argyros
Professor, Committee Member

Asterios Leonidis
Assist. Professor, Committee Member

Departmental approval:

Grigorios Tsagkatakis
Assist. Professor, Director of Graduate Studies

Heraklion, March 2025

A Real-Time Physics Framework, with advanced Cutting, Tearing and Piercing interactions for mobile XR applications

Abstract

Extended Reality(XR) has demonstrated great potential for use in various fields across the industry. While graphics have advanced, reaching an acceptable level even in mobile XR Head Mounted Displays(HMDs), physics simulations lack behind. Traditional physics frameworks, like PhysX, are mainly focused on rigidbody simulations and often exclude support for softbodies and real-time XR interactions like cutting, tearing and piercing, crucial for improving realism. Although some frameworks support these advanced interactions, they require intense computational power, not available in mobile XR HMDs.

In this thesis, we introduce a lightweight, custom physics framework, called XR-PBD (Extended Reality - Position Based Dynamics), that addresses these limitations. This framework leverages the efficiency of particle physics and the stability of Extended Position Based Dynamics (XPBD) to support real-time simulations. A number of constraints, such as Distance, Volume, Shape Matching, Collinear and Insertion, are used for simulating softbodies, rigidbodies, cloths, ropes and stiff rods. Additional optimization techniques, like advanced memory management, SIMD (Single Instruction Multiple Data) instructions and parallel programming further reduce the computational needs, allowing our framework to function even in devices of lower CPU and GPU specifications, such as mobile XR HMDs. Finally, to support advanced user interactions our system introduces novel geometric algorithms and tools that enhance evaluation of real-time cutting, tearing and piercing on tetrahedral meshes.

Performance metrics in the context of realistic use cases prove that, our framework is fully capable of simulating complex scenarios, with thousands of particles and constraints, in real-time. This underscores our framework's potential to democratize complex real-time physics simulations with advanced user interactions in low-performance XR devices.

Ένα πλαίσιο προσομοίωσης φυσικής σε πραγματικό χρόνο, με προηγμένες αλληλεπιδράσεις κοπής, σχισίματος και διάτρησης για φορητές εφαρμογές εκτεταμένης πραγματικότητας

Περίληψη

Η Εκτεταμένη Πραγματικότητα (XR) έχει αποδείξει μεγάλη δυναμική για χρήση σε διάφορους τομείς της βιομηχανίας. Παρόλο που τα γραφικά έχουν εξελιχθεί, φτάνοντας σε ένα υψηλό επίπεδο ακόμα και σε φορητές συσκευές Εκτεταμένης Πραγματικότητας, οι φυσικές προσομοιώσεις υστερούν. Τα παραδοσιακά πλαίσια φυσικής, όπως το PhysX, εστιάζουν κυρίως σε προσομοιώσεις άκαμπτων σωμάτων και συχνά δεν υποστηρίζουν μαλακά σώματα και αλληλεπιδράσεις σε πραγματικό χρόνο, όπως κοπή, σχίσιμο και διάτρηση, που είναι κρίσιμες για τη βελτίωση του ρεαλισμού. Ακόμα και αν κάποια πλαίσια υποστηρίζουν αυτές τις προηγμένες αλληλεπιδράσεις, απαιτούν υψηλή υπολογιστική ισχύ, η οποία δεν είναι διαθέσιμη σε φορητές συσκευές Εκτεταμένης Πραγματικότητας.

Στην παρούσα διατριβή, παρουσιάζουμε ένα ελαφρύ, προσαρμοσμένο πλαίσιο φυσικής, που αποκαλούμε XR-PBD XR-PBD (Extended Reality - Position Based Dynamics) και αντιμετωπίζει αυτούς τους περιορισμούς. Αυτό το πλαίσιο την αποδοτικότητα της φυσικής με σωματίδια και τη σταθερότητα του Extended Position Based Dynamics (XPBD) για την υποστήριξη προσομοιώσεων σε πραγματικό χρόνο. Ένα σύνολο περιορισμών, για απόσταση, όγκο, διατήρηση σχήματος, συνευθειακότητα και διάτρηση, χρησιμοποιούνται για την προσομοίωση μαλακών και άκαμπτων σωμάτων, υφασμάτων, σχοινιών και άκαμπτων ράβδων. Επιπλέον, τεχνικές βελτιστοποίησης, όπως προηγμένη διαχείριση μνήμης, εντολές SIMD (Single Instruction Multiple Data) και παράλληλος προγραμματισμός, μειώνουν περαιτέρω τις υπολογιστικές απαιτήσεις, επιτρέποντας στο πλαίσιο μας να λειτουργεί ακόμα και σε συσκευές με χαμηλές προδιαγραφές CPU και GPU , όπως οι φορητές συσκευές Εκτεταμένης Πραγματικότητας. Τέλος, για την υποστήριξη προηγμένων αλληλεπιδράσεων χρήστη, το σύστημά μας εισάγει νέους γεωμετρικούς αλγορίθμους και εργαλεία που βελτιώνουν την αξιολόγηση της κοπής, του σχισίματος και της διάτρησης σε τετραεδρικά πλέγματα σε πραγματικό χρόνο.

Οι μετρήσεις απόδοσης στο πλαίσιο ρεαλιστικών περιπτώσεων χρήσης αποδεικνύουν ότι το πλαίσιο μας είναι πλήρως ικανό να προσομοιώνει σύνθετα σενάρια, με χιλιάδες σωματίδια και περιορισμούς, σε πραγματικό χρόνο. Αυτό υπογραμμίζει τη δυνατότητα του πλαισίου μας να εκδημοκρατίσει τις σύνθετες προσομοιώσεις φυσικής σε πραγματικό χρόνο με προηγμένες αλληλεπιδράσεις χρήστη ακόμα και σε φορητές συσκευές Εκτεταμένης Πραγματικότητας χαμηλών προδιαγραφών.

Aknowledgments

First and foremost, I would like to express my sincere gratitude to my supervisor, Prof. George Papagiannakis, for his invaluable guidance and unwavering support throughout this journey.

I am also deeply appreciative of Dr. Manos Kamarianakis, Dr. Antonis Protopsaltis, and Nikos Lydatakis for their insightful feedback and the countless brainstorming sessions that greatly contributed to this work.

Furthermore, I would like to extend my thanks to my colleagues and co-workers for their support and collaboration.

Lastly, I am profoundly grateful to my family and my girlfriend for their unwavering encouragement and immense support throughout this endeavor.

Knowledge is not a destination,
but an endless journey of discovery.

Contents

List of Tables

List of Figures

Chapter 1

Publications arisen from this thesis

In this section we present a short overview of our previous publications related to this work.

Paul Zikas, Antonis Protopsaltis, Nick Lydatakis, Mike Kentros, Stratos Geronikolakis, Steve Kateros, Manos Kamarianakis, Giannis Evangelou, Achilleas Filippidis, Eleni Grigoriou, Dimitris Angelis, Michail Tamiolakis, Michael Dodis, George Kokiadis, John Petropoulos, Maria Pateraki, and George Papagiannakis. MAGES 4.0: Accelerating the world's transition to VR training and democratizing the authoring of the Medical Metaverse. IEEE Computer Graphics and Applications, 43(2):43–56, 2023.

In this work, we propose MAGES 4.0, a novel software development kit to accelerate the creation of collaborative medical training applications in virtual/augmented reality (VR/AR). Our solution is essentially a low-code metaverse authoring platform for developers to rapidly prototype high-fidelity and high-complexity medical simulations. MAGES breaks the authoring boundaries across extended reality, since networked participants can also collaborate using different VR/AR as well as mobile and desktop devices, in the same metaverse world. With MAGES we propose an upgrade to the outdated 150-year-old master-apprentice medical training model. Our platform incorporates, in a nutshell, the following novelties: 1) 5G edge-cloud remote rendering and physics dissection layer, 2) realistic real-time simulation of organic tissues as soft-bodies under 10 ms, 3) a highly realistic cutting and tearing algorithm, 4) neural network assessment for user profiling and, 5) a VR recorder to record and replay or debrief the training simulation from any perspective.

Manos Kamarianakis, Antonis Protopsaltis, Dimitris Angelis, Michail Tamiolakis, and George Papagiannakis. Progressive Tearing and Cutting of Soft-bodies in High-performance Virtual Reality. The Eurographics Association, 2022. ISSN: 1727-530X.

In this work, we present an algorithm that allows a user within a virtual environment to perform real-time unconstrained cuts or consecutive tears, i.e., progressive, continuous fractures on a deformable rigged and soft-body mesh model in high-performance 10ms. In order to recreate realistic results for different physically-principled materials such as sponges, hard or soft tissues, we incorporate a novel soft-body deformation, via a particle system layered on-top of a linear-blend skinning model. Our framework allows the simulation of realistic, surgical-grade cuts and continuous tears, especially valuable in the context of medical VR training. In order to achieve high performance in VR, our algorithms are based on Euclidean geometric predicates on the rigged mesh, without requiring any specific model pre-processing. The contribution of this work lies on the fact that current frameworks supporting similar kinds of model tearing, either do not operate in high-performance real-time or only apply to predefined tears. The framework presented allows the user to freely cut or tear a 3D mesh model in a consecutive way, under 10ms, while preserving its soft-body behaviour and/or allowing further animation.

Manos Kamarianakis, Antonis Protopsaltis, Michail Tamiolakis, and George Papagiannakis. Realistic soft-body tearing under 10ms in VR, May 2022. arXiv:2205.00914 [cs].

In this publication, we present a novel integration of a real-time continuous tearing algorithm for 3D meshes in VR, suitable for devices of low CPU/GPU specifications, along with a suitable particle decomposition that allows soft-body deformations on both the original and the torn model.

Manos Kamarianakis, Nick Lydatakis, Antonis Protopsaltis, John Petropoulos, Michail Tamiolakis, Paul Zikas, and George Papagiannakis. "Deep Cut": An all-in-one Geometric Algorithm for Unconstrained Cut, Tear and Drill of Soft-bodies in Mobile VR, August 2021. arXiv:2108.05281 [cs].

In this work, we present an integrated geometric framework: "deep- cut" that enables for the first time a user to geometrically and algorithmically cut, tear and drill the surface of a skinned model without prior constraints, layered on top of a custom soft body mesh deformation algorithm. Both layered algorithms in this frame- work yield real-time results and are amenable for mobile Virtual Reality, in order to be utilized in a variety of interactive application scenarios. Our framework dramatically improves real-time user experience and task performance

in VR, without pre-calculated or artificially designed cuts, tears, drills or surface deformations via predefined rigged animations, which is the current state-of-the-art in mobile VR. Thus our framework improves user experience on one hand, on the other hand saves both time and costs from expensive, manual, labour-intensive design pre-calculation stages.

Chapter 2

Introduction

In recent years, Extended Reality (XR) has transformed various fields, including simulations, gaming, training, and education [34][16]. While advancements in graphics have achieved high levels of visual fidelity, even in applications designed for untethered head-mounted displays (HMDs), physics simulations have lagged behind, often neglected due to their inherent complexity. This disparity results in applications that lack realism and fail to deliver a fully immersive experience.

Existing physics simulation frameworks, discussed further in Chapter 3, are often built on proprietary, closed-source platforms with limited device compatibility[1]. Moreover, these frameworks predominantly focus on rigid-body simulations[32], lacking support for soft deformable objects and advanced user interactions like cutting, tearing and suturing that are crucial in fields like medicine. Even when frameworks do offer these features, they are frequently unoptimized for XR devices or lack developer-friendliness, requiring advanced programming expertise to operate effectively[9]. Additionally, while these frameworks may expose real-time simulation data, they often do so in formats that are cumbersome and challenging to work with.

To address these challenges, this thesis proposes, a custom physics simulation framework, called XR-PBD (Extended Reality - Position Based Dynamics), tailored for performance-constrained devices such as standalone XR head-mounted displays (HMDs). Leveraging the power and stability of Extended Position Based Dynamics (XPBD) our method enables real-time simulation of rigidbodies, softbodies, cloths, ropes and stiff-rods. Designed, with low-performance HMD devices in mind, it utilizes a massively parallel architecture, optimized for multi-core CPUs. Built around Unity Engine, it offers easy to use, no-code simulation editing and authoring tools to assess the accuracy of user interactions, allowing even inexperienced developers to use it.

Let us delve deeper into the rationale behind our choices when designing the framework.

2.1 Why XPBD?

All simulations methods fall into three general categories, force-based, impulse-based and position-based simulations [27].

Force-based simulation methods, like mass-spring systems[17], resolve interpenetrations and enforce constraints by applying forces, often modeled as springs. While these methods are considered the most physically accurate, as they simulate natural forces acting on objects, they are computationally expensive and prone to instability due to oscillations, which can lead to numerical issues.

Impulse-based simulation methods [20][3][2] directly modify the velocities of objects to resolve interpenetrations and constraints. They achieve this by applying impulses, which are instantaneous changes in velocity during a single time step. While these methods are computationally simpler than force-based approaches, they are less accurate and can suffer from issues such as object drifting and jittering due to the way collisions and constraints are handled.

Position-based methods [4][5] operate directly on object positions, resolving interpenetrations by instantly moving colliding objects apart within a single time step. Constraints are enforced similarly, by adjusting positions iteratively. These methods are computationally efficient and straightforward to implement. However, they are even less physically accurate than velocity-based methods. Despite this limitation, their speed and stability make them ideal for real-time large-scale simulations.

Given the performance constraints of mobile HMDs, a position-based simulation method was the most suitable choice. Although position-based approaches are less accurate than force- or velocity-based alternatives, the errors are imperceptible to the naked eye, justifying the trade-off for speed and efficiency.

The original position-based simulation method, Position-Based Dynamics (PBD) [22], offers unconditional stability while maintaining low computational cost. Although PBD is fast, robust and simple, it has shortcomings that make it hard to work with. One key limitation is that constraint stiffness is time-step dependent, making it difficult and time-consuming to tune it.

Extended Position-Based Dynamics (XPBD) [18] is a modern position-based simulation method designed to address the constraint tuning challenges present in the original PBD algorithm. By introducing compliance, XPBD makes stiffness a time-step-independent parameter while preserving PBD's efficiency and stability.

Similarly to PBD, XPBD employs an iterative constraint-solving approach, resolving constraints sequentially. It offers flexibility by allowing the number of solver substeps and constraint iterations to be adjusted, enabling developers to balance computational efficiency and simulation accuracy based on the application's performance constraints. This adaptability, is another key fact in choosing XPBD over traditional simulation methods.

2.2 Why Unity Engine?

In order to visualize simulations we needed a game engine with a fast and easy to use interface and with a large community supporting it.

Unity Engine [30] is one of the most known and widely used game engines, recognized for its versatility and comprehensive feature set. Designed for the development of professional applications, Unity provides a developer-friendly editor that simplifies the creation process across a wide variety of target devices. Its architecture emphasizes extensibility, enabling developers to create and integrate third-party packages. These packages enhance the development experience by adding specialized functionalities or providing ready-to-use assets.

In recent years, Unity has become a leading choice for developing XR applications and games. This is largely due to its ease of use, robust support for a wide range of XR head-mounted displays (HMDs), and an active developer community. Unity's built-in tools and frameworks significantly streamline the development process, allowing for rapid prototyping and deployment of XR applications with existing boilerplate code.

Additionally, Unity not only provides a comprehensive set of tools for developing XR-PBD without the need for low-level optimizations but also includes an advanced editor layer. It features a SIMD-accelerated mathematics library, parallel job execution, and the Burst compiler, which translates high-level C# scripts into highly optimized assembly. Lastly, its robust editor API enables the creation of intuitive, GUI-based developer tools for efficiently configuring various aspects of our simulations.

2.3 Why CPU?

In XR-PBD we opted to use CPU over GPU, primarily due to resource allocation and hardware limitations.

In most applications, designed for portable HMDs, the GPU is heavily utilized for rendering at high frame rates, leaving little to no room for additional calculations. Offloading physics to the GPU would introduce a bottleneck, reducing frame rates and increasing latency. Additionally, many standalone HMDs have limited support for Compute Shaders rendering GPU implementations for them almost impossible.

On the other hand, CPUs in these devices provide multi-core processing and can efficiently handle parallel execution with support of SIMD instructions, ensuring a real-time performance without affecting graphical fidelity.

Chapter 3

Related Work

3.1 Physics Engines

In this section we will explore previous work related to physics simulation frameworks, as well as their limitations.

3.1.1 Nvidia PhysX

Nvidia PhysX [1] is one of the most widely utilized physics engine middleware solutions in the game industry. Initially developed for use with PhysX PPUs, it later transitioned to GPU acceleration while retaining a CPU fallback mechanism. PhysX primarily employs a velocity-based solver for accurate rigid body dynamics, complemented by an additional position-based solver, which is utilized in scenarios requiring increased stability, such as cloth and softbody simulations.

Unity Engine integrates PhysX as the underlying physics engine while providing its own high-level API for developers. This implementation includes support for rigid bodies, joints, and cloth simulation; however, it lacks native support for softbody physics.

Despite its robustness, PhysX relies on CUDA[25] for hardware acceleration, which is not readily available on mobile HMDs. Furthermore, the Unity implementation of PhysX does not offer built-in support for real-time cutting, tearing, or suturing—features that are essential for highly immersive simulations.

3.1.2 Unity Physics

Unity physics [32] is a fairly new physics engine, developed for the Unity Data Oriented Technology Stack (DOTS). It is a massively parallel, completely deterministic, stateless physics engine that runs on the CPU and takes advantage of the Burst compiler.

Under the hood, it uses a velocity based solver, to apply impulses in order to correct constraints. This leads to instabilities and simulation explosions, when multiple forces act on the same object. As described in their official page, "Unity

Physics was designed to support developers who do not necessarily need a fully-featured physics package like PhysX, but instead want a simpler subset of features that they can freely control and customize". With this in mind, the framework does not have built-in support for sotfbody dynamics or advanced user interactions like cutting and tearing, but expects the developer to implement them by extending the modular framework core.

3.1.3 Nvidia Flex

Nvidia flex [26] is a unified particle physics framework. It models all type of bodies as a set of particles and uses a position based solver (Extended Position Based Dynamics) to solve the physics equations. Making use of Extended Position Based Dynamics it is unconditionally stable and fast, with support for soft bodies, rigid bodies, fluids and smoke.

On the downside, it uses CUDA [25] to accelerate calculations on the GPU, which is a proprietary framework, only supported with Nvidia graphics cards. Moreover, recently Nvidia has stopped updating Flex Framework, in favor of the newer versions of PhysX, which is better for the average market needs, that focus on rigidbody simulations. Finally, this framework does not provide advanced interactions like cutting and tearing either.

3.1.4 Obi Physics

Obi Physics [29] consists of a set of 4 different packages, Obi Softbody, Obi Fluid, Obi Rope, Obi Cloth, that all together create a fully featured physics framework. Obi physics is written from the ground up as a Unity Package and it makes use of the Burst Compiler as well as GPU Compute shaders.

Obi is an easy to use engine but is focused mainly on softbody, rope, cloth and fluid simulations and does not provide advanced user interactions like piercing and cutting, as provided in XR-PBD.

3.1.5 SOFA Framework

The SOFA (Simulation Open Framework Architecture) [9] is an open-source, flexible platform designed for real-time physical simulation, particularly in research and development contexts. It provides a modular architecture for creating and testing simulations involving complex physical interactions, such as in biomechanics, robotics, and virtual reality. Developers can combine and customize components for tasks like collision detection, soft body dynamics, and finite element modeling.

Although, SOFA is a fully featured simulation framework with a modular design, a complex setup process and development pipeline is required to make it work. Scenes need to be described in a proprietary SOFA Scene format with a text editor and previewed with the provided GUI. While Unity Plugin for SOFA exists, it is complex to set up, has minimal support, and can only be used for Rendering a SOFA scene in unity or modifying core properties of the simulation like

gravity. Additionally, since the framework is designed for research applications, it focuses on extendability rather than performance. Finally, it is not compatible with Android, which is the OS used by most HMDs.

On the other hand, XR-PBD is provided as a standalone Unity package, simplifying the setup process. Using Unity editor API we also provide reusable components with simplified editor GUIs to help author a simulation, requiring no additional proprietary format, but doing it directly from the Game Engine's scene. Support for multiple Operation Systems is also provided directly from the Game Engine.

3.1.6 iMSTK

The iMSTK (interactive Medical Simulation Toolkit) [21] is an open-source simulation platform based on Position Based Dynamics (PBD). It offers extensive real-time simulations, featuring continuous collision detection, smooth particle hydrodynamics, integrated haptics and compatibility with Unity and Unreal, among others.

While this framework supports realtime physics simulations and advanced interactions like piercing, it lacks in developer experience being written in C++ and having a minimal Unity Integration, usually requiring coding to achieve usable results.

On the other hand XR-PBD provides a no-code editor to author simulations by reusing modular components. Furthermore, we expand on the advanced user interactions provided with realtime tearing and cutting.

Table 3.1: Features of each framework. (✓) fully supported, (✘) partially supported, (✗) not supported

Name	**Rigids**	**Softs**	**Cloths**	**Ropes & Rods**	**Cut Tear Pierce**	**Cross Platform**	**No-code**	**Performance**
Unity PhysX	✓	✗	✓	✗	✗	✓	✓	High
Unity Physics	✓	✗	✗	✗	✗	✓	✓	High
NVIDIA Flex	✓	✓	✓	✓	✘	✗	✘	High
Obi Physics	✗	✓	✓	✓	✘	✓	✓	High
SOFA	✓	✓	✓	✘	✓	✗	✘	Low
imstk	✓	✓	✓	✘	✓	✗	✘	Medium
Ours	✓	✓	✓	✓	✓	✓	✓	High

3.2 Advanced 3D Mesh Operations: Cutting and Tearing

In this section we will explore previous work related to real time advanced user interactions such as cutting and tearing.

3.2.1 Progressive Tearing and Cutting of Soft-bodies in High-performance Virtual Reality

In our earlier work [14][15][13], we presented a framework for real-time tearing and cutting of soft bodies in Virtual Reality (VR). The simulation of soft, deformable meshes was achieved using a spring-mass model and surface meshes. A bounded plane served as a blade, and its interaction with the surface mesh was used to define a cutting path. The triangles of the surface mesh were split based on the intersection points, and new particles were introduced along the cut to enhance collision handling and minimize artifacts. This framework was later enhanced with additional tools and was embedded in our VR Training SDK [34] to further improve object interactions.

Although our previous algorithm achieved real-time performance even on low-performance mobile head-mounted displays (HMDs), the quality of the cuts was suboptimal. Furthermore, as the simulation was limited to the surface, the cuts lacked depth, making the approach unsuitable for volumetric objects. In this thesis, we aim to build upon our earlier work by extending the algorithm to 3D space and tetrahedral models. We will apply this enhanced approach to deformable meshes based on the proposed XPBD framework.

3.2.2 Arbitrary Cutting of Deformable Tetrahedralized Objects

Sifakis et al. [28] introduced an algorithm for cutting deformable tetrahedral meshes that employs a dual-mesh approach, utilizing a low-resolution simulation mesh for physics calculations and a higher-resolution render mesh for visualization. Their method identifies intersections between a triangulated cutting surface and both the simulation and render meshes. Each tetrahedron in the simulation mesh is associated with a set of render mesh triangles contained within it. During cutting, new virtual nodes are generated at intersection points, forming the basis for retriangulating the render mesh. Instead of splitting intersected tetrahedra, the algorithm duplicates them to prevent the formation of sliver and ill-conditioned tetrahedra that produce instabilities in the simulation, a common issue in traditional mesh-splitting techniques. Deformation is simulated using the vertices of the simulation mesh, while collision detection relies on the triangles of the render mesh's outer surface.

While their algorithm offers an efficient method for cutting tetrahedralized deformable objects, its complexity makes implementation challenging and real-time performance difficult to achieve for reasonably sized meshes. This issue is even

more pronounced on low-performance devices like mobile HMDs, where maintaining two duplicate mesh structures significantly impacts performance. Additionally, as noted in their paper, using the render mesh for collision detection can be problematic due to the presence of small, ill-conditioned triangles.

To alleviate these issues, in our framework, we avoid using two separate mesh structures. Instead, we create the render mesh based on the existing, underlying simulation mesh. Problems with ill-conditioned tetrahedra generated after a cut, are resolved by performing a clean-up operation to completely remove them.

Chapter 4

Our Framework - XR-PBD

In this chapter we will provide a short introduction on Position Based Dynamics (PBD) and Extended Position Based Dynamics (XPBD), we will describe our framework, XR-PBD (Extended Reality - Position Based Dynamics), and highlight its key features, novelties and design decisions.

4.1 Background

In the following sections, we will be presenting a general background perspective Position Based Dynamics (PBD) and Extended Position Based Dynamics (XPBD), but refer to [22], [5] and [18] for more information.

4.1.1 Position Based Dynamics (PBD)

Position-Based Dynamics (PBD) [22] is a simulation method designed for interactive applications, where stability and performance are prioritized over strict physical accuracy. Unlike traditional physics-based methods that solve differential equations to compute forces or impulses and integrate them into velocities and positions, PBD operates directly on positions, which eliminates drifting or instabilities when a solution is not reached. The algorithm enforces constraints iteratively to maintain the system's physical properties.

Given a system of particles with positions x_i, masses m_i, velocities v_i, constraints $C_j(x_1, x_2, ..., x_n) = 0$ and a time step dt, simulation starts by calculating the velocity that should be applied to each particle based on external forces $vext_i$. Then an initial prediction of the new positions x_i^* is calculated, typically with $x_i^* = x_i + v_i + vext_i$. For each constraint solving iteration, adjustments are calculated using constraint gradients ∇C_j and are distributed to the affected particles proportionally to their inverse masses $w_i = \frac{1}{m_i}$.

Then the position delta Δx_i for the particle p_i for each constraint is given by:

$$\Delta x_i = -k\, s\, w_i\, \nabla_{x_i} C(x_1, ..., x_n)$$

, where

$$s = \frac{C(x_1, ..., x_n)}{\sum_{j=1}^{n}(w_j|\nabla_{x_j} C(x_1, ..., x_n)|^2)}$$

and $k \in [0, 1]$, representing the stiffness of the constraint. Finally, at the end of the above iterations, the velocities of the particles are updated with $v_i = \frac{x_i^* - x_i}{dt}$. This process repeats for the specified number of substeps.

4.1.2 Extended Position Based Dynamics (XPBD)

Position Based Dynamics (PBD) has the reputation of being not physical and inaccurate when it comes to the stiffness of the constraints. It is dependent on the time step size, making the constraints stiffer when smaller time steps are used.

Extended Position Based Dynamics (XPBD), extends the aforementioned method and eliminates this issue, by removing k and adding an extra term $\frac{\alpha}{dt^2}$ to the denominator of the scalar s:

$$s = \frac{C(x_1, ..., x_n)}{\sum_{j=1}^{n}(w_j|\nabla_{x_j} C(x_1, ..., x_n)|^2) + \frac{\alpha}{dt^2}}$$

where x_i is the particle's position, $C(x_1, ..., x_n)$ is the constraint, $w_i = \frac{1}{m_i}$ the inverse mass of each particle and a the constraint compliance- the time step independent inverse of the physical stiffness.

4.2 Parallel Sucessive Over Relaxation (SOR) Solver

Typically, Position Based Dynamics use a Gauss-Seidel solver, calculating each constraint's correction, applying it and then continuing to the next one. This is a linear process and does not take advantage of modern CPUs where multiple physical threads can execute at the same time. To take advantage of multithreading we have used a Jacobi solver in which all the corrections of the constraints are summed to a temporary array. Then, at the end of each constraint solving iteration, the average position delta is added back to the predicted position.

Even though Jacobi solvers convert the solution to a massively parallel one, they are characterized by slower convergence due to the corrections not being propagated immediately. To address this issue various hybrid solutions[11] have been proposed utilizing graph coloring, in which the constraints are split into independent sets. Each set contains only constraints that do not affect the same particles. Sets containing a lot of constraints are solved in parallel with Gauss Seidel, while the remaining smaller sets are merged and solved together with a Jacobi solve.

Although the graph coloring approach typically converges faster with fewer overall iterations, we opted for a Jacobi Solver for all constraints due to its simplicity and the need to support real-time cutting and tearing operations. These

dynamic simulation mesh modifications introduce or remove constraints at runtime, invalidating the graph structure and requiring frequent recalculations of independent sets.

While the standard Jacobi Solve ensures convergence by averaging constraint corrections, it can sometimes be overly aggressive, requiring additional iterations to reach a stable solution. To mitigate this, we adopt a strategy similar to that of *Miles Macklin et al. [19]*. Specifically, we introduce a global per-constraint-type, factor $s \in [1, 2]$ that scales the average constraint correction $\Delta x'_{avg_i} = s\Delta x_{avg_i}$, effectively controlling the rate of successive over-relaxation. Sucessive Over Relaxation factor s is empirically defined by the developer, based on the simulated scene. The impact of it in the simulation is shown in Figure 4.1.

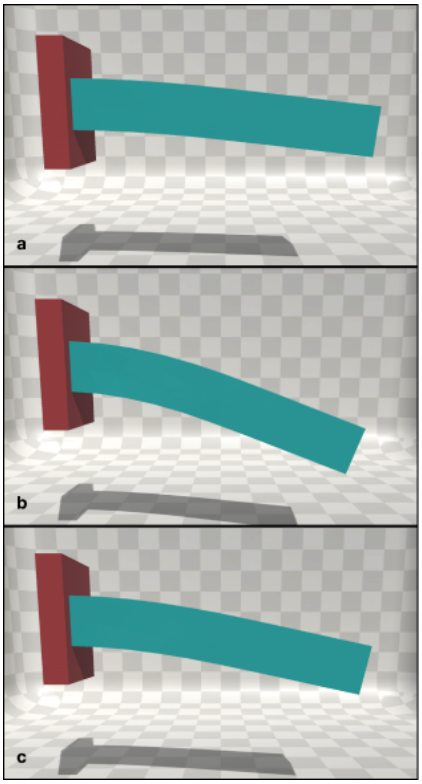

Figure 4.1: Successive Over-Relaxation(SOR) on a beam simulation. a) Ground truth, b) 3 substeps, 4 constraint solver iterations, SOR Factor 1. c) 3 substeps, 4 constraint solver iterations, SOR Factor 1.8.

Furthermore, constraints are processed in priority groups based on their type, with corrections applied to the predicted positions immediately after a group is solved. This approach accelerates error propagation, improving the overall convergence speed. For more details on our SOR solver, refer to section 4.2.2, where the algorithm is presented.

4.2.1 Particles

XR-PBD employs particles as the fundamental building blocks for all object types. Particles are chosen for their simplicity and versatility in representing various object shapes. Particles are point masses, with a sphere collision shape, on which positional changes are applied based on constraints. By using particles exclusively to construct collision shapes, we reduce the complexity of collision pair implementations and accelerate collision detection through the use of uniform spatial hashing grids, as detailed section 5.3.4. Moreover, particles offer fine-grained control and inherently support massive parallelism, enhancing computational efficiency.

4.2.2 Main Simulation Loop

The primary simulation loop in our SOR Solver, manages the integration of the physics simulation for each time step. First the Collision World is updated and the contact pairs between colliding particles are collected. The updated velocities are then computed, after which damping is applied and the new particle positions are predicted. Subsequently, constraints are resolved, and the average position correction, scaled by the relaxation factor, is applied to the predicted particle positions. Finally, the updated velocity of each particle is determined, which dictates whether the particle enters a sleeping state or adopts the predicted position.

Algorithm 1 Main Simulation Loop

1: $UpdateCollisionWorld()$
2: $CollectContactPairs()$
3: $\Delta t_s \leftarrow \Delta t/n$
4: **for** n substeps **do**
5: **for all** particles **in parallel do**
6: update velocity $v \leftarrow v + g * \Delta t_s$
7: apply damping $v \leftarrow d * v$
8: predict position $x^* \leftarrow x + v * \Delta t_s$
9: **for all** constraint types **do**
10: $SolveConstraintType()$
11: **for all** particles **in parallel do**
12: calculate updated velocity $v \leftarrow (x^* - x)/\Delta t_s$
13: apply positions or sleep
14: **if** $v > sleepThreshold$ **then**
15: $x \leftarrow x^*$

Algorithm 2 SolveConstraintType() Function

def *SolveConstraint*():
for n iterations **do**
  for all particles p **in parallel do**
    initialize position accumulation $acc_p = 0$
  for k constraints affecting particle p **in parallel do**
    calculate correction dx_p
    add to accumulated correction $acc_p \leftarrow acc_p + dx_p$
  for all particles p **in parallel do**
    calculate average correction $\Delta x_{avg_p} = acc_p / k_p$
    scale correction by SOR factor $\Delta x_{avg_p} s * \Delta x_{avg_p}$
    find new particle position $x_p^* = x_p + \Delta x_{avg_p}$

4.2.3 Particle Sleeping

Similar to the work presented by *Miles Macklin et al[19]*, to reduce drifting caused by constraints not fully satisfied at the end of a timestep, we introduce sleeping. After each simulation step, before applying the predicted position for each particle we check if the velocity of them has dropped below a certain threshold. In that case, we discard the predicted position and freeze the particle in place.

4.3 Constraints

Constraints of type $C_j(x_1, x_2, ..., x_n) = 0$ are used to form positional relationships between particles. After solving them we acquire a positional change Δx in the gradient direction, which contributes to the particle's position at the end of the timestep. Multiple type of constraints are utilized by the Actors to simulate different material properties. In this section, we will briefly describe each constraint formulation, but refer to Section 5.5 for a deep dive into the implementation details.

4.3.1 Distance Constraint

In order to keep particles a certain distance apart, we have used a distance constraint as proposed by *Jakobsen et al[12]*:

$$C(x_1, x_2) = |x_1 - x_2| - d$$

where x_1,x_2 the particle positions and d the rest length of the constraint.

Calculating the constraint corrections for each particle we get:

$$dx_1 = -\frac{w_1}{w_1 + w_2}(|x_1 - x_2| - d)\frac{x_1 - x_2}{|x_1 - x_2|}$$

$$dx_2 = +\frac{w_2}{w_1 + w_2}(|x_1 - x_2| - d)\frac{x_1 - x_2}{|x_1 - x_2|}$$

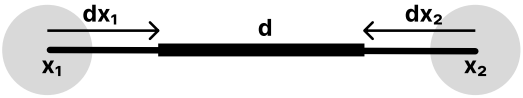

Figure 4.2: The distance constraint between two particles at positions x_1, x_2 and the corrections dx_1 and dx_2 for each particle.

4.3.2 Collision Constraint

In XR-PBD framework, collisions are resolved using dynamically created collision constraints. These constraints are represented as infinitely stiff distance constraints, but are only resolved when the distance between the two affected particles is smaller than $2r$, where r is the particle radius.

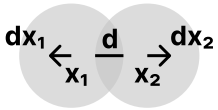

Figure 4.3: The collision constraint between two particles at positions x_1, x_2, the interpenetration distance d, and the corrections dx_1 and dx_2 for each particle.

4.3.3 Volume Constraint

Our models typically consist of tetrahedron structures. In order to constraint the volume of the model, we use Tetrahedral Volume Constraints. This type of constraint, is responsible for the retention of a tetrahedron volume across timesteps. Instead of using the constraint function of $[C(x_1, x_2, x_3, x_4) = V - V_{rest}$, we opted to use the constraint function:

$$C(x_1, x_2, x_3, x_4) = 6(V - V_{rest})$$

where x_1, x_2, x_3, x_4 the positions of the particles forming the tetrahedron, V the volume of the tetrahedron and V_rest the volume of it at the rest configuration. The constant 6 is used to cancel out unnecessary divisions introduced by the tetrahedron volume defined as $V = \frac{1}{6}((x_2 - x_1) \times (x_3 - x_1)) \cdot (x_4 - x_1)$

The corrections for the above constraint are calculated as:

$$\Delta x_i = s * w_i * \nabla C_i$$

, where

$$s = \frac{6(V - V_rest)}{\sum_{i=1}^{4}(w_i|\nabla_i C|^2) + \frac{\alpha}{dt^2}}$$

with gradients:

$$\nabla_{x_1} C = (x_4 - x_2) \times (x_3 - x_2)$$
$$\nabla_{x_2} C = (x_3 - x_1) \times (x_4 - x_1)$$
$$\nabla_{x_3} C = (x_4 - x_1) \times (x_2 - x_1)$$
$$\nabla_{x_4} C = (x_2 - x_1) \times (x_3 - x_1)$$

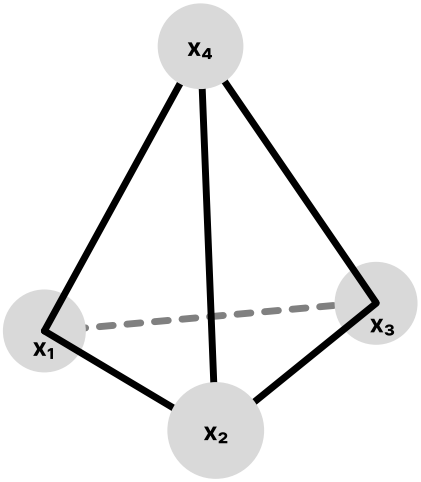

Figure 4.4: The particles at positions x_1, x_2, x_3, x_4 forming a tetrahedron, used for a volume constraint.

4.3.4 Shape Matching Constraint

Shape Matching Constraints [23] are commonly employed for simulating both rigid bodies and soft bodies, though they do not inherently preserve volume. This constraint type assigns each particle a target position, which the solver attempts to achieve during simulation.

The constraint is described by:

$$C(x) = |x - x_{target}|$$

, where x is the particle position and x_{target} is the position the constraint tries to achieve.

4.3.5 Collinear Constraint

To maintain three particles in alignment by minimizing the angle formed between the segments connecting them, we introduce the Collinear Constraint. The constraint is defined as

$$C(x_1, x_2, x_3) = acos(n_1 \cdot n_2)$$

where $n_1 = x_2 - x_1$ and $n_2 = x_3 - x_2$

For the Collinear Constraint the gradients used to find the final correction for each particle are:

$$\nabla_{x_1} C = \frac{n_2 - n_1(n_1 \cdot n_2)}{|x_1 - x_3|}$$

$$\nabla_{x_2} C = \frac{n_1 - n_2(n_1 \cdot n_2)}{|x_2 - x_3|}$$

$$\nabla_{x_3} C = \nabla_{x_1} - \nabla_{x_2}$$

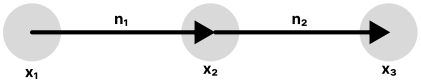

Figure 4.5: The collinear constraint between three particles at positions x_1, x_2, x_3.

4.3.6 Insertion Constraint

To support real-time piercing interactions we propose a new type of constraint, the Insertion Constraint. It involves 5 particles from which 3 form the triangle that is punctured at a point q with barycentric coordinates $u = (u_x, u_y, u_Z)$, and the other 2 form a line segment, that is part of the needle used for piercing, from now on referred to as "needle segment". The constraint is described as:

$$C(x_1, x_2, x_3, n_1, n_2) = |q - r|$$

where:

$$q = u_x x_1 + u_y x_2 + u_z x_3$$

and

$$r = n_1 + t(n_2 - n_1)$$

, where t is the point along the needle segment the intersection happens.

To derive the constraint gradients we will assume that t is constant between timesteps. Although this is not completely true, t has insignificant change between timesteps, small enough that the error this produces is not visible in the final result and does not justify the extra computational complexity solving with t as variable adds.

The gradient for each particle, as calculated are shown below:

$$\nabla_{x_1} C = \frac{q - r}{|q - r|} u_x$$

$$\nabla_{x_2} C = \frac{q - r}{|q - r|} u_y$$

$$\nabla_{x_3} C = \frac{q - r}{|q - r|} u_z$$

$$\nabla_{n_1} C = \frac{q - r}{|q - r|}(t - 1)$$

$$\nabla_{n_2} C = -\frac{q - r}{|q - r|}t$$

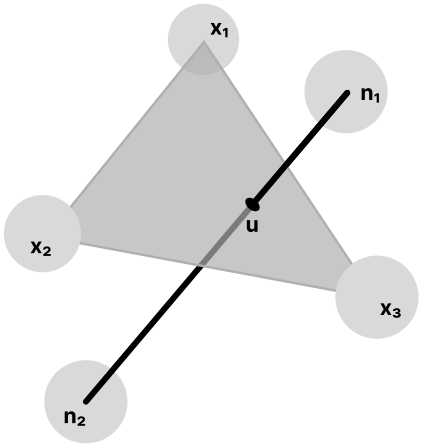

Figure 4.6: An insertion constraint involving a triangle formed by the particles at positions x_1, x_2, x_3 and the needle segment formed by particles at n_1, n_2. The point u is the puncture point described with triangle barycentric coordinates.

4.4 Actors

To eliminate memory fetch operations and improve performance, we implement a data structure similar to an Entity Component System (ECS), further described in Section 5.3. In doing so, challenges are introduced, especially for inexperienced developers. Accessing or modifying the physics properties of a body requires complex operations. Additionally, adding or removing bodies to the physics world requires deep knowledge of memory management operations.

To alleviate these issues and to allow inexperienced developers to use XR-PBD, we introduce the concept of Actors. Actors are simple Components that expose an object-oriented developer API for underlying complex operations. In addition, actors are responsible for managing the physics simulation properties of an object as well as managing the rendering of it. Lastly, actors provide an editor layer allowing developers to author simple simulations directly from Unity Editor, without the need of scripting. For further details on each Actor type, refer to Section 5.7.

4.5 Advanced User Interactions

To support advanced user interactions, such as real-time cutting, tearing and piercing, we introduce a set of novel algorithms based on geometric intersections and

new constraint types.

4.5.1 Cutting

XR-PBD introduces two cutting algorithms for softbodies: Cutting by Removing Volume Constraints and Cutting by Splitting Volume Constraints. Both techniques rely on a cutting disc to detect intersections with the underlying simulation mesh.

When a cut operation is initiated, the algorithm first identifies intersections between the cutting disc and the volume constraints of the softbody. Depending on the selected method:

- **Cutting by Removing Volume Constraints:** The detected constraints are entirely removed, allowing the mesh to separate naturally.
- **Cutting by Splitting Volume Constraints:** Instead of removal, the intersected volume constraints are split in half and re-tetrahedralized based on the intersection points, resulting in a more precise incision.

Following the cut operation, a second pass is executed to refine the mesh. This step eliminates residual distance constraints and sliver tetrahedra that may form ill-conditioned volume constraints, ensuring simulation stability.

Additionally, XR-PBD provides a dedicated cutting algorithm for ropes and rods. In this case, the cutting disc detects intersections not with volume constraints, but with the distance and collinear constraints that define the rope or rod. Once detected, the intersected constraints are removed, and the render mesh is rebuilt to reflect the cut. For an in-depth explanation and implementation details refer to Section 5.7.3.3.

4.5.2 Tearing

In XR-PBD, tearing is seamlessly integrated into the constraint-solving process. After computing the positional corrections for each particle, we estimate the force exerted on them using the equation $F = \frac{\Delta x}{\Delta t}$, where Δx represents the positional correction applied to the particle, and Δt is the simulation timestep. If the calculated force surpasses a predefined threshold, the corresponding constraint is deactivated, effectively simulating the tearing effect. This approach ensures that tearing emerges naturally from the simulation, responding dynamically to stress forces rather than relying on predefined fracture patterns. For an in-depth explanation and implementation details refer to Section 5.7.3.2.

4.5.3 Piercing

To enable real-time piercing in softbodies, we introduce a novel method based on Insertion Constraints. In this approach, the piercing rigid object is represented as a discrete curve, where individual particles are positioned and constrained using

shape-matching constraints to maintain structural integrity. During simulation, the system continuously detects intersections between this curve and the external faces of the softbody mesh. Once an intersection is identified, Insertion Constraints are established, ensuring that the piercing object follows a consistent trajectory through the softbody, adhering to the initial insertion path. To enhance realism, frictional forces are incorporated by applying corrective adjustments to the affected particles. These corrections depend on the relative velocity between the softbody and the piercing object, preventing unrealistic sliding and ensuring a physically plausible interaction. For an in-depth explanation and implementation details refer to Section 5.7.2.2.

4.6 Detectors

To enable real-time monitoring of changes in the physics objects, we introduce the concept of "Detectors." These components simplify the process of identifying events such as cuts, tears, punctures, or collisions within the simulation mesh. Instead of requiring the end-users of XR-PBD to develop complex algorithms for such detections, we provide a set of intuitive, ready-to-use detectors that expose events directly. These events can be used to trigger custom behaviors or processes based on user actions and simulation changes.

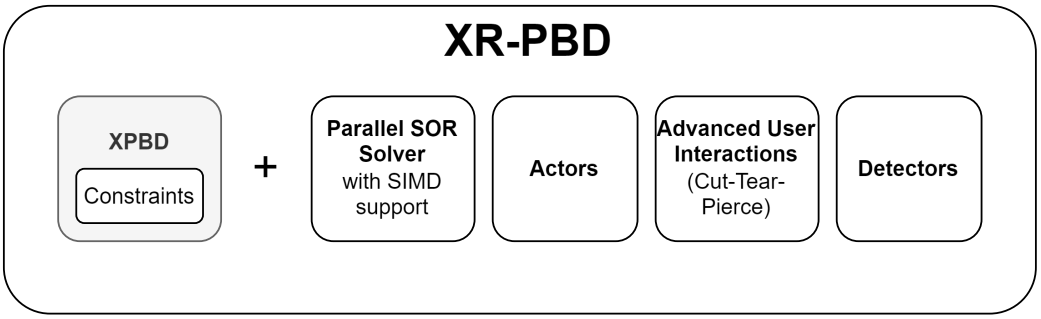

Figure 4.7: The main components of XR-PBD.

Chapter 5

Implementation

This chapter provides an in-depth examination of the implementation of XR-PBD, detailing its fundamental principles and the optimization strategies employed. Although the rendering components are closely integrated with the Unity Engine and are discussed in that context, the core physics framework is designed to function independently, requiring only minimal modifications.

5.1 Tetrahedral Mesh

To store information about volumetric meshes we have created a serializable Tetrahedral Mesh object. It contains information about the vertex positions and the indices of the vertices that form each tetrahedron. A UV coordinate for each vertex is also stored for texture mapping, in case our mesh contains this information. Additionally, we store information of the indices that belong in each sub-mesh to allow for multi-material tetrahedral meshes.

When generating such a mesh, an automatic background process is triggered to compute the neighbors of each tetrahedron. These neighbor relationships are stored in an array to optimize future neighbor search operations. Additionally, a render mesh is generated for preview purposes. To construct this render mesh, we include all tetrahedral faces that do not share a neighbor on that side. The resulting mesh is then displayed in a small preview scene with panning functionality, and a screenshot is captured to serve as a thumbnail in the Unity Editor (Figure 5.1).

To easily create a Tetrahedral Mesh, we have created a set of tools that the developer can use. Below we will explain their functionality and how they work.

5.1.1 Importing a Tetrahedral Mesh

To import tetrahedral meshes, we have created a custom Scriptable Importer that parses MSH files and stores them in our Data Structures. This allows the developer

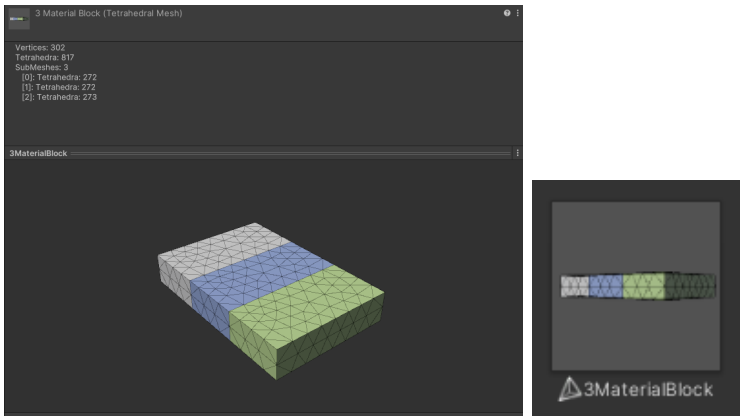

Figure 5.1: The tetrahedral mesh preview scene and thumbnail

to create highly detailed tetrahedral meshes with custom parameters and predefined sub-meshes, using the graphic, easy to use interface of a dedicated software, and then import them for XR-PBD.

Prior to importing the mesh, we give the developer the ability to make minor adjustments to the scale, multiplying all the node positions provided in the file with a scale factor. This transforms the object from the scale used in the program it was created in, to a suitable scale consistent with other objects in the scene.

5.1.2 Converting Surface Mesh to Tetrahedral Mesh

Alternatively, for simple watertight surface models, with only one sub-mesh, we have created a built-in tetrahedralizer (Figure 5.3), that directly converts them to a tetrahedral mesh. To achieve this we have implemented an Editor Tool that uses Delaunay Tetrahedralization to convert any given watertight surface mesh with no self intersections, into a Tetrahedral Volumetric Mesh. This mesh is then stored as an MSH file, which is then imported with the aforementioned importer. An example with a conversion of a beam from surface to tetrahedral mesh can be seen in Figure 5.2

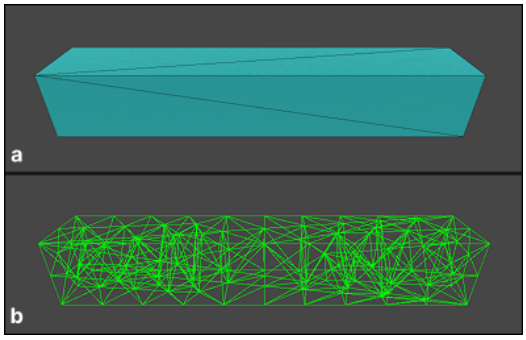

Figure 5.2: A beam converted from a surface mesh (a) to a tetrahedral mesh (b).

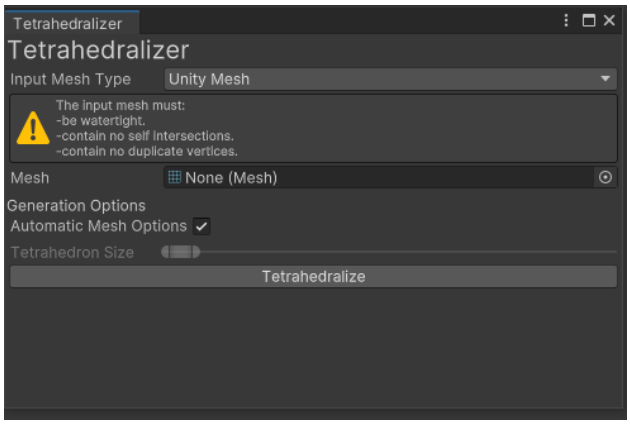

Figure 5.3: The tetrahedralizer GUI

5.2 Simulation Mesh

The simulation mesh structure holds information for the physical representation of a model. It stores both particle data and the constraints that connect them. Based on the tetrahedral mesh and the desired physical properties different constraint types are used. Additionally, the simulation mesh provides an API for accessing real-time simulation data, including particle dynamic properties and constraints, which are stored and internally managed by the physics world.

To facilitate the creation of a simulation mesh from a tetrahedral mesh, we provide a Converter Tool. This tool generates a particle at each vertex position and additional constraints based on the desired physics body type.

For softbody generation, the tool calculates the length of each tetrahedral edge and creates distance constraints with the corresponding rest lengths. Additionally, a volume constraint is assigned to each tetrahedron in the original mesh. For rigidbody creation, a shape-matching constraint is applied to each particle, while for other material types the developer can combine constraint types to achieve the desired physics behavior.

During the conversion process, developers can fine-tune the physical properties of each constraint. When a tetrahedral mesh with multiple sub-meshes is used as input, the developer is given the option to assign different physical properties to each sub-mesh.

Figure 5.4: The simulation mesh converter GUI

5.3 Physics World

At the heart of XR-PBD lies the physics world component. This component hosts the logic for the simulation loop and manages the preallocation of a large contiguous memory pool, which is subsequently utilized to store particle and built-in constraint data for the simulation meshes added to it. This information is stored as large arrays -one for each constraint type- from which a continuous index range is given to each simulation mesh. Furthermore, the physics world maintains and updates the collision world- a spatial hashing grid designed for efficient detection

of particle-particle contacts.

5.3.1 Adding a Simulation Mesh

To add a simulation mesh to the physics world, we allocate the necessary memory space for its particle data and constraints from the preallocated continuous memory block of the physics world. The particle data and built-in constraint data are then copied from the serialized file on disk into the reserved memory block.

If the simulation mesh is expected to change dynamically by adding particles or constraints at runtime, additional memory is reserved at the end of the allocated block. When the available memory is fully utilized by newly added elements, a complete memory restructuring occurs, doubling the size of the memory block for that simulation mesh, and rearranging nearby data. Throughout this process, simulation mesh data is kept in adjacent memory cells to minimize cache misses and enhance performance.

5.3.2 Removing a Simulation Mesh

When a body is destroyed from the Unity scene, it is also marked for destruction in the physics world. After every physics step of the simulation loop, we check for objects that are marked as destroyed and we remove them from the physics world's internal memory. Removing objects, causes memory fragmentation, leaving empty memory space between the data, resulting in degraded performance. To address this issue, when a simulation mesh is removed we rearrange the array cells, to place the remaining data in contiguous memory space. An example of this operation is shown in Figure 5.5.

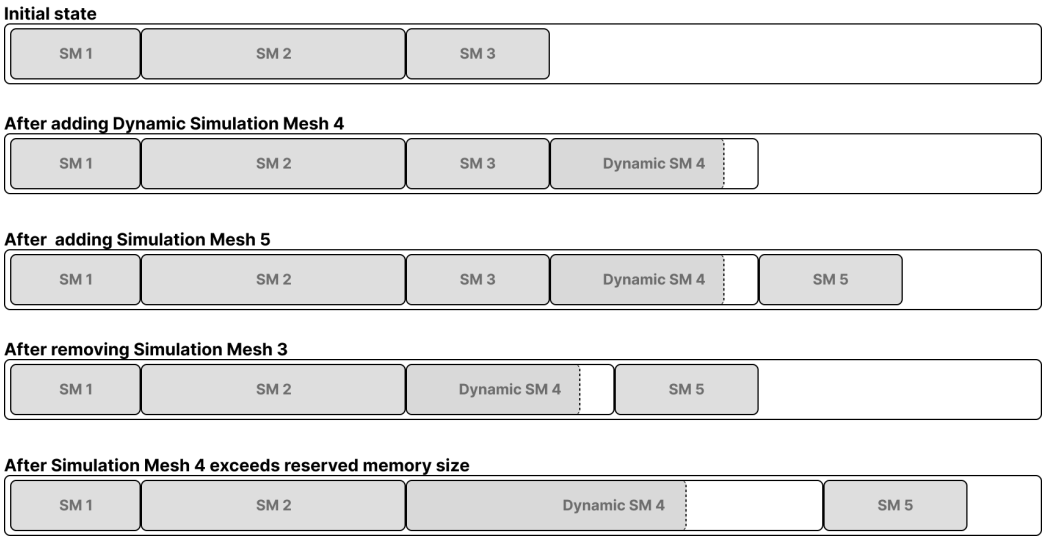

Figure 5.5: Physics World Data Array layout during various add/remove operations.

5.3.3 Accessing Simulation Mesh properties

Due to constant changes in the memory layout, accessing the data of the simulation meshes is done with handles (ids), similar to an Entity Component System (ECS) architecture. When a simulation mesh is added to the physics world it is given a handle. The handle is then mapped to a struct containing the indices for each array, where the data for this simulation mesh can be found. The indices are updated when the memory layout is updated.

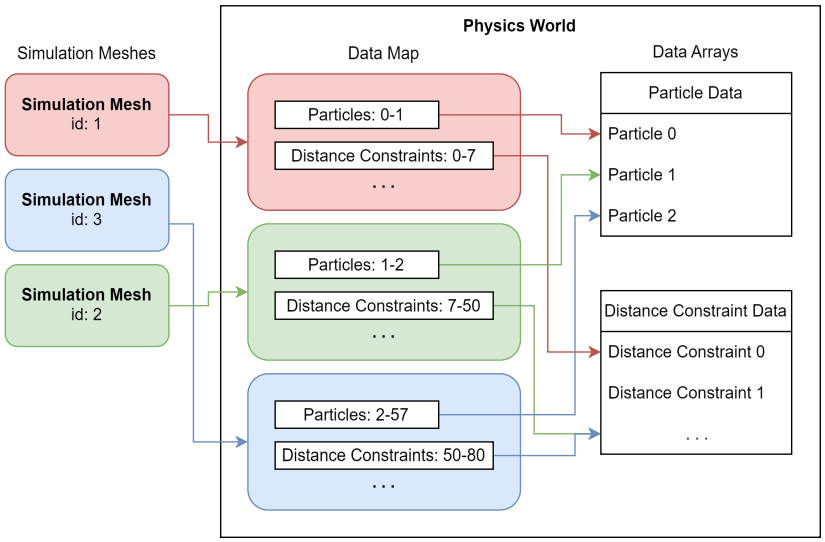

Figure 5.6: Accessing Physics World Simulation data.

5.3.4 Collision World

Naive approaches to collision detection involve checking every particle against all other particles with collisions enabled, requiring n^2 comparisons for n particles. This method becomes computationally expensive as the number of particles increases, resulting in poor performance and making it impractical for real-time applications.

To address this challenge, we implemented a parallel uniform spatial hash grid [8] to efficiently identify particle-particle contact pairs. This grid divides the simulation space into uniform cubic cells, where each cell has a side length twice the particle collision radius. At the beginning of each solver loop, particles are grouped based on the cell they occupy. When detecting potential collisions for a given particle, we only consider particles within the same cell or adjacent cells. This drastically reduces the number of pairwise comparisons, lowering the computational complexity. Using parallel processing, this method ensures rapid and scalable collision detection, enabling real-time performance even with large particle systems.

5.4 Particles

As previously discussed, our framework is fundamentally particle-based. In this approach, particles serve as point masses, each represented by a spherical collision shape, to which constraint corrections are applied during simulation.

During the conversion of a tetrahedral mesh into a simulation mesh, particles are strategically positioned at the vertices of the tetrahedra. This ensures that all physical interactions and deformations are governed by the particle-based constraint system, allowing for efficient and stable simulations.

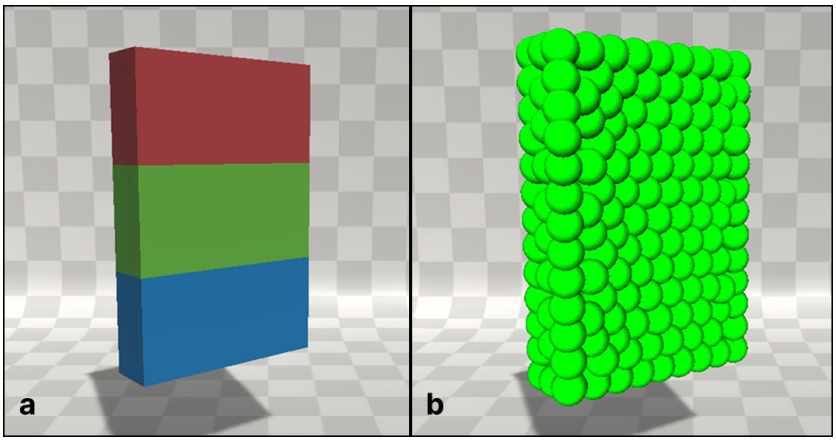

Figure 5.7: Particles placement. a) The original object. b) The particles placed on it.

Internally, in the solver, the particle representation consists of the following properties:

```
struct Particle
{
    float3 position;
    float3 velocity;
    float inverseMass;

    bool enabled;
    int phase;
    int physicsMaterialIndex;
}
```

Figure 5.8: The particle representation

In addition to standard dynamic properties such as position, velocity, and inverse mass, XR-PBD incorporates an enabled flag to temporarily deactivate

particles when not in use, as well as a phase index to group particles into distinct categories. These groups facilitate control over interactions, such as disabling collisions between specific particle groups. Lastly, each particle is associated with a physics material index, which references its corresponding physics material- a concept discussed in greater detail in the following sections.

5.5 Constraints

In this section we will deep-dive into the implementation details and the memory layout of the constraints we previously discussed.

5.5.1 Distance Constraints

Our distance constraint representation consists of the following properties:

```
struct DistanceConstraint
{
    int particleA;
    int particleB;

    float restLength;

    bool enabled;
    bool breakable;
    int physicsMaterialIndex;
}
```

Figure 5.9: The Distance Constraint representation

In addition to the common distance constraint properties, we have added a flag for enabling/disabling the constraint, a flag for marking it as breakable and an index pointing to the physics material from which the physics properties for this constraint are taken.

5.5.2 Collision Constraints

As described previously, collision constraints are used to resolve particle-particle interpenetration. They are similar to distance constraints, but are only evaluated and solved when $d < 2 * r$, where r is the particle radius. Bounciness may be added to collisions, by multiplying the calculated corrections by a number > 1 before applying them to the particle positions.

Additionally, to allow the users to create collision groups, we utilize the phase of the particles. The first bit in the phase is used as a flag for self-collision, while

the rest form a collision layer. When iterating across collected collision pairs in order to resolve them, we ignore all pairs with particles belonging to the same collision layer, if the self-collision flag is not set.

5.5.3 Volume Constraints

The representation of the Volume Constraint in our solver consists of the following properties:

```
struct VolumeConstraint
{
    int particleA;
    int particleB;
    int particleC;
    int particleD;

    float restLength;

    bool enabled;
    bool breakable;
    int physicsMaterialIndex;
}
```

Figure 5.10: The Volume Constraint representation

In addition to the common volume constraint properties, such as the particles forming a tetrahedron and the volume at rest configuration, we have added a flag for enabling/disabling the constraint, a flag for marking it as breakable and an index pointing to the physics material from which the physics properties for this constraint are taken.

5.5.4 Shape Matching Constraints

Our Shape Matching Constraint representation consists of the following properties:

```
struct ShapeMatchingConstraint
{
    int particle;

    float3 targetPosition;

    bool enabled;
    int physicsMaterialIndex;
}
```

Figure 5.11: The Shape Matching Constraint representation

In addition to the usual shape matching constraint properties, such as the particle for the constraint and the target position, we have added a flag for enabling/disabling the constraint and an index pointing to the physics material from which the physics properties for this constraint are taken.

5.5.5 Collinear Constraints

Our Collinear Constraint representation consists of the following properties:

```
struct CollinearConstraint
{
    int particleA;
    int particleB;
    int particleC;

    bool enabled;
    int physicsMaterialIndex;
}
```

Figure 5.12: The Collinear Constraint representation

In addition to the collinear constraint properties, such as the particles of the constraint, we have added a flag for enabling/disabling the constraint and an index pointing to the physics material from which the physics properties for this constraint are taken. We have opted to not add any target angle property, since we only use the constraint for rods, which are always using collinear constraints with 180° as target angle. Instead, this property is used as a constant in the solver.

5.5.6 Insertion Constraints

Our Insertion Constraint representation contains the following properties:

```
struct InsertionConstraint
{
    int particleA;
    int particleB;
    int particleC;

    int particleNeedleA;
    int particleNeedleB;

    float3 triangleBarycentricCoords;
    float needleInterpolationFactor;

    bool enabled;
}
```

Figure 5.13: The insertion constraint representation

In addition to the insertion constraint properties, such as the particles forming the triangle, the particles of the needle segment and the barycentric coordinates of the intersection, we have added a flag for enabling/disabling the constraint.

Instead of adding this constraint to the builtin types, we opted to add it as a User Defined Constraint by using the API described in the following section. This was decided since the constraint was used only in very specific cases and adding it to the built-in constraints would add performance overhead, even in simulation scenarios that did not use it.

5.5.7 User Defined Constraint Types

In addition to the built-in constraints, XR-PBD provides an API that allows developers to define custom constraints and seamlessly integrate them into the solver loop without modifying the core program. These custom constraints are solved last, after the built-in constraints, following a specified priority order. Developers can also configure parameters such as solver iteration count and the successive over-relaxation factor to tailor the behavior of their constraints to specific requirements.

5.6 Physics Materials

In many simulation frameworks, additional properties for constraints and particles, such as compliance or friction, are typically stored within the respective data structures for each constraint or particle. However, in XR-PBD, instead of embedding this information directly into every constraint or particle entry in the data array, we use a dedicated Physics Material struct. Each constraint or particle entry references this struct through an index.

This design choice reduces the size of individual constraint and particle data entries, thereby minimizing cache misses during runtime and improving overall performance. Moreover, modifying the physics behavior at runtime is efficient, as it only requires updating the corresponding material property itself, rather than modifying multiple data entries. Additionally, modifying the behavior of an individual particle or constraint is faster, as it only requires changing the material index of it rather than changing all the physics properties. Finally, materials facilitate grouping particles or constraints with shared properties, allowing for simpler assignment and management of custom behaviors.

Our physics material representation contains the following properties:

```
struct PhysicsMaterial
{
    // Particle Properties
    float ParticleDamping;
    float StaticFriction;
    float KineticFriction;

    // Constraint Properties
    float DistanceConstraintCompliance;
    float VolumeConstraintCompliance;
    float CollinearConstraintCompliance;
    float ShapeMatchingConstraintCompliance;

    // Tear Properties
    public float TearForceThreshold;
}
```

Figure 5.14: The physics material representation.

5.7 Actors

In this section, we will deep dive in the implementation details of each Actor type included in the XR-PBD framework.

5.7.1 Rigidbody

Rigidbody Actors are designed to represent rigid objects that experience only minor deformations. In traditional simulation frameworks, rigidbodies are typically represented using a position and rotation for their center of mass along with a predefined collision shape. In contrast, XR-PBD adopts a particle-based representation for rigidbodies, combined with a shape matching simulation approach [23]. This unified representation simplifies the solver architecture and streamlines collision detection by treating all object types consistently.

To construct the simulation mesh for rigidbodies, we assign one Shape Matching Constraint to each particle, with a dynamically adjusted target position, changing during simulation. After predicting the new positions of the particles and prior to solving the shape matching constraints, we calculate the new center of mass for the rigidbody. This is achieved by computing the weighted average of the particle positions, using their respective masses as weights.

Next, to determine the rigidbody's rotation, we compute the best-fit rotation for the updated particle configuration relative to the original shape. This is done at the new center of mass using a covariance matrix. Once the center of mass position and rotation are determined, we calculate the target position for each Shape Matching Constraint. The target position is obtained by applying the initial offset of each particle from the center of mass to the current center of mass, ensuring the rigidbody maintains its structural integrity.

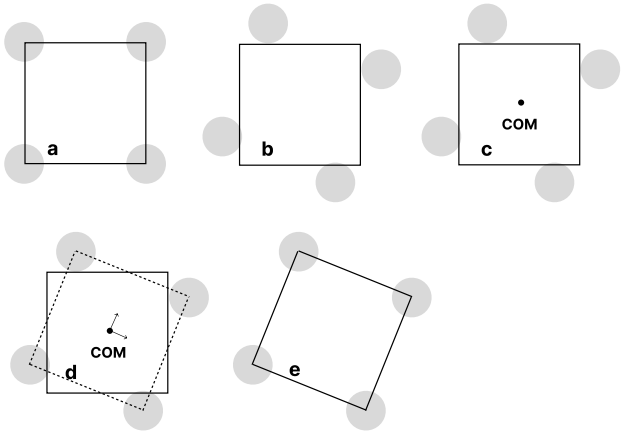

Figure 5.15: Rigidbody simulation steps. a) Initial state of rigidbody. b) Particles positions are predicted. c) New Center of Mass (COM) position is calculated. d) New Center of Mass (COM) rotation is calculated. e) The rigidbody mesh is updated.

5.7.1.1 Rendering

To render the rigidbodies we use the calculated Center Of Mass position and rotation. Before the simulation starts, we store the offset the mesh origin has from

the center of mass. At runtime, after each time step, to find the new mesh position and rotation we apply the offset we calculated to the new center of mass position and rotation.

5.7.2 Softbody

The Softbody actor is responsible for managing and rendering soft, deformable meshes. These meshes are composed of Distance and Volume Constraints, with a Distance Constraint assigned to each unique edge of the tetrahedrons and a Volume Constraint for each tetrahedron. This combination ensures the conservation of an object's volume and internal structure. Developers can control the softness or "squishiness" of the body by adjusting the constraint compliance parameter α, where a lower value results in a stiffer material.

5.7.2.1 Rendering

To enable the visualization of high-resolution meshes without compromising performance, we decoupled the visible mesh from the simulation mesh. During the actor creation process at edit time, we establish a mapping between the high-resolution render mesh provided by the developer and the low-resolution simulation mesh. Each vertex of the render mesh is associated with the tetrahedron in which it is embedded, and its barycentric coordinates are stored in an array. For vertices not enclosed within any tetrahedron, the nearest tetrahedron is identified and used for mapping.

At runtime, after each simulation time step, the render mesh is updated based on the deformed simulation mesh. The new position of each render mesh vertex is computed using the updated positions of the particles that form the corresponding mapped tetrahedron and the precomputed barycentric coordinates.

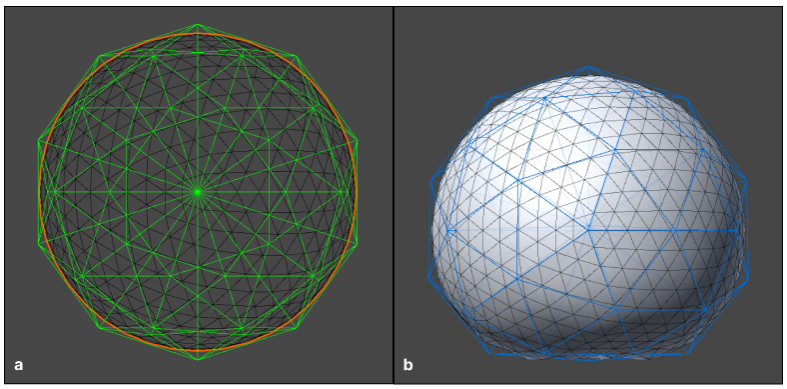

Figure 5.16: Softbody rendering. a) Initialization. The low-resolution simulation mesh (green color) and the high-resolution render mesh (black color). b) Deformed state. The low-resolution simulation mesh (blue color) and the shaded high-resolution render mesh (white faces, black lines).

5.7.2.2 Piercing

XR-PBD introduces additional constraints to enable piercing interactions with watertight softbody objects. To simulate the piercing process, we utilize a rigidbody referred to as the "needle," which is created by placing a series of particles along a curve that approximates the shape of the needle. These neighboring particles are connected to form the needle segments, with the first particle in the simulation mesh representing the needle tip.

Before the simulation begins, we identify and store all the exterior triangle faces of the volume constraints that define the softbody. These faces serve as reference points for detecting intersections during the simulation.

At runtime, after each simulation timestep, we check for intersections between the needle segments and the faces of the tetrahedra defined by the volume constraints. For each segment, we start from the tip and move backward, checking for potential intersections with the softbody. If a segment intersects with the softbody and the previous segment was also intersecting in the previous timestep, the intersection is considered valid. Otherwise, it is ignored. Valid intersections result in the creation of an insertion constraint, which involves the three particles that form the intersected triangle, as well as the two particles of the needle segment.

To prevent incorrect collisions between the needle and softbody particles when the needle is inside the softbody, we disable collision detection between them. To detect when a needle particle has entered the softbody, we perform swept sphere tests that check for intersections with the exterior faces of the softbody, which were stored before the simulation started. If a collision is detected and the final position of the particle is on the opposite side of the triangle's normal, the collision is disabled for that particle. If the particle later intersects a triangle again (as detected by another swept sphere test) and the final position is on the same side of the normal, the collision is re-enabled. Additionally, the collision of the first of the needle particles that are outside of the volume of the softbody is also disabled to prevent collisions while the user is trying to force the needle in.

It is possible, usually with low iteration count, that the insertion constraints may not be fully satisfied at the end of a timestep. In that case there is a risk that, if the needle is pulled sideways with extreme force, no intersection with the triangle occurs anymore and the constraint is incorrectly removed as shown in figure 5.18. To mitigate this issue, we do not immediately remove the insertion constraint if no intersection is detected. Instead, we wait for one of the two particles of the needle segment to pierce through the plane defined by the intersected triangle. This is achieved using the same swept sphere test, avoiding any additional computational complexity.

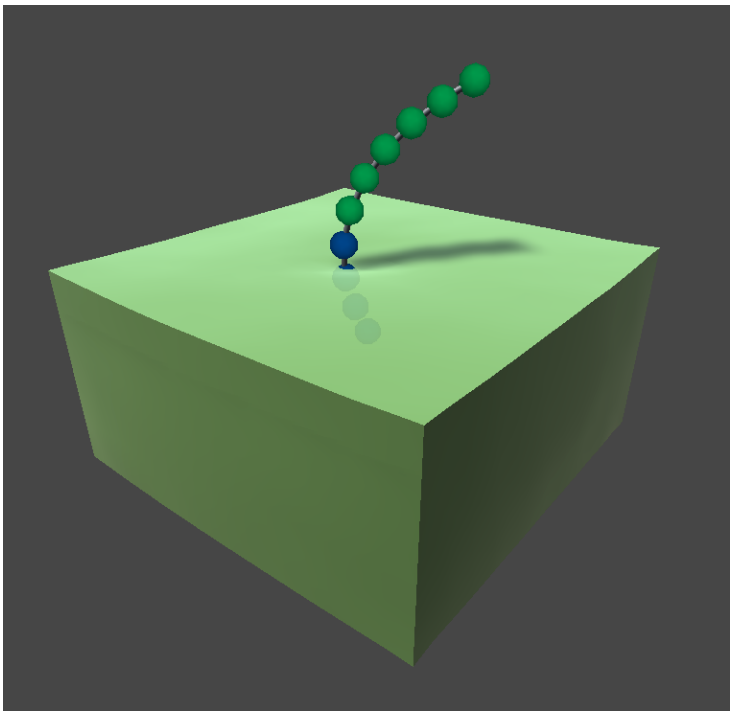

Figure 5.17: Collisions of particles that are inside the pierced softbody and the next particle from the ones outside are disabled (shown with blue).

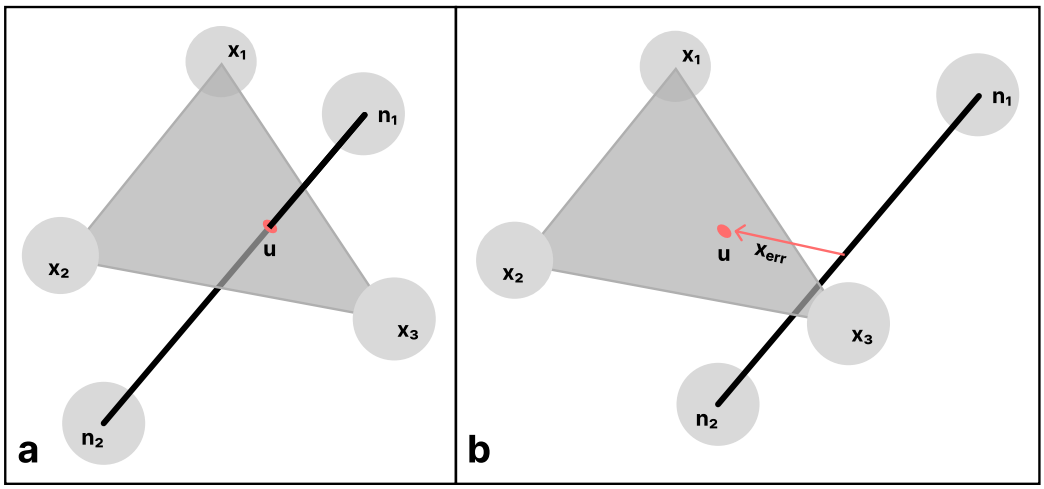

Figure 5.18: Removing the insertion constraint. a) The insertion constraint when added. b) The insertion constraint at the end of the time step is not satisified and it is incorrectly removed when needle segment-triangle intersection is used.

To restrict the free movement of the needle through the softbody, we incorporate friction constraints. These constraints are applied alongside the insertion

constraints and serve to reduce the relative movement between the insertion point u and the needle, governed by a pre-defined friction coefficient.

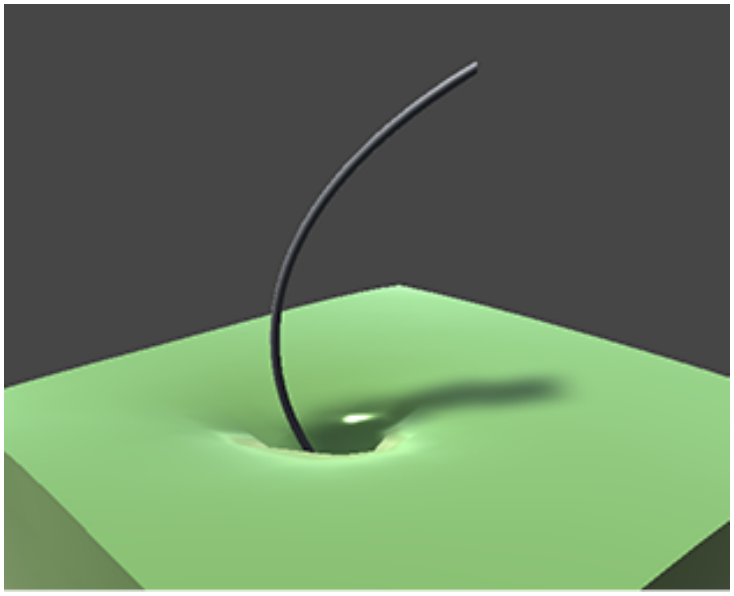

Figure 5.19: High friction resists the needle's penetration into the softbody.

5.7.3 Dynamic Softbody

Dynamic softbody actor, behaves similary to the softbody actor mentioned above, using Distance constraints at Tetrahedra edges and Volume constraints for each tetrahedron to maintain volume. As far as rendering is concerned, unlike sofbody actor, dynamic softbody actor does not bind a render mesh but rather creates one from the underlying simulation mesh structure.

5.7.3.1 Rendering

Before the simulation starts, we preprocess the simulation mesh, calculating all the neighbors of each tetrahedron and storing them to an array. For each tetrahedron face we check if any neighboring tetrahedra are available and if the volume constraints for them are enabled in the simulation mesh. For the trianglular faces that meet these conditions we create new vertices and faces for the render mesh. In case we want to smooth shade the object, instead of creating new vertices for every face, we first check if there are any already created vertices at that point and reuse them instead. In that case, the normal vectors for each vertex are calculated by averaging the normal vectors of the faces at each point.

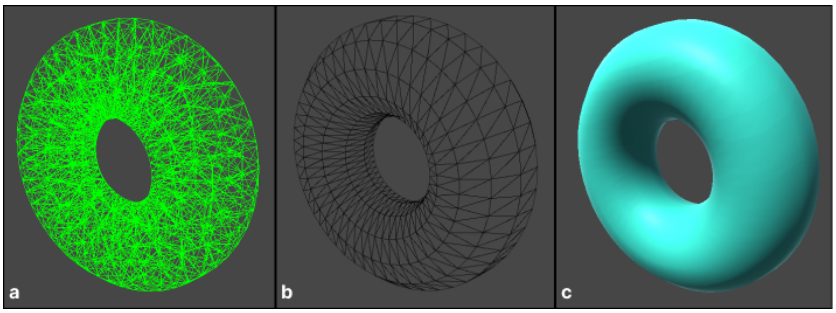

Figure 5.20: Generating render mesh based on underlying Simulation Mesh. a) The underlying torus Simulation Mesh. b) The generated faces for the render mesh. c) The final smooth shaded render mesh.

If the simulation mesh consists of multiple physics materials, we can divide the triangles into distinct sub-meshes during the creation of the render mesh. This approach enables the developer to assign different render materials to each sub-mesh, providing greater visual customization.

In cases where the developer opts to use transparent materials, additional considerations are necessary. Specifically, we generate faces for tetrahedra located at the boundaries between different physics materials. Even though these tetrahedra may have neighboring elements, their faces can become visible due to the transparency of the render materials, as illustrated in Figure 5.21. This ensures that the rendered mesh maintains visual consistency and accurately reflects the material boundaries.

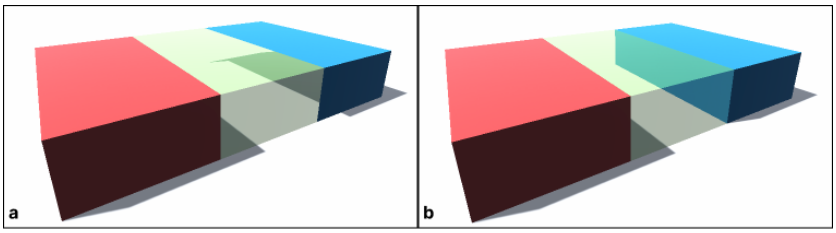

Figure 5.21: Boundary face generation for transparent meshes. a) Without boundary faces. b) With boundary faces.

5.7.3.2 Tearing

XR-PBD provides realtime tearing of dynamic softbodies, based on forces acting on them. This means that these actors can be torn by pulling them appart or creating extreme deformations. By setting the breakable flag on the underlying Volume and Distance constraints of a Dynamic Softbody, force acted on them is monitored and if it exceeds a tear threshold value defined in the Physics Material, these constraints are disabled. After each timestep the Dynamic Softbody Actor checks the updated flag of the simulation mesh which is set by the physics world

when any changes to it have been made during the last time step. If this flag is set a render mesh rebuild is triggered.

Sometimes when a volume constraint gets broken, the distance constraints at the edges of it may not break due to force at them not being large enough. This will result in "ghost forces" in the simulation, making the object behave as connected in the tear area, even though visually it appears completely torn. To mitigate this, after a tear we traverse the simulation mesh and remove any distance constraints that do not resemble an edge of any volume constraints.

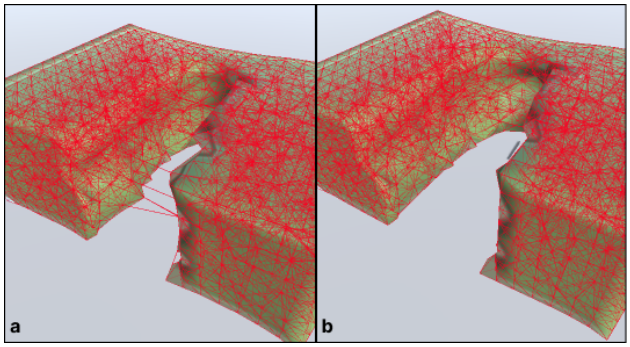

Figure 5.22: Distance Constraints cleaning after a tear. a) Without cleaning, distance constraints remain, spanning across the torn area. b) After distance constraint cleaning, the residual constraints are removed.

To improve even more the torn object both visually and simulation-wise, we implemented a volume constraint cleaning algorithm to remove small unconnected pieces after a tear. To do so, after each timestep we traverse the simulation mesh and remove any volume constraint with one or no neighbors. These volume constraints are loosely attached or are usually small separated pieces. This further cleans up the remaining mesh, reducing artifacts at the torn area.

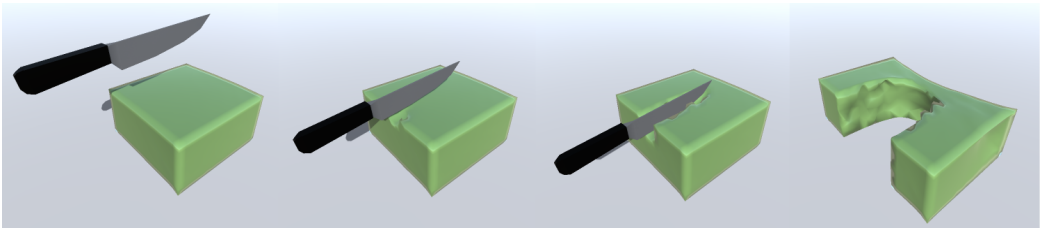

Figure 5.23: Tearing a slime with a knife and then stretching it.

5.7.3.3 Cutting

In scenarios requiring precise incisions, the tearing algorithm may be insufficient. To address this, XR-PBD offers a more advanced and precise cutting algorithm in

addition to tearing. Unlike tearing, which relies on force, the cutting algorithm is based solely on geometric intersections and does not consider physical properties.

We provide two distinct cutting algorithms: one optimized for lower precision with minimal computational overhead, and another designed for high precision, but more complex resulting in a linear increase in computational overhead with each successive cut. This flexibility allows developers to select the approach that best suits their application's requirements for accuracy and performance.

Cutting by Removing Volume Constraints
The first cutting method is designed for scenarios involving frequent cuts and operates by completely removing tetrahedra based on their intersection with a cutting disc, similar to the tearing process.

The cutting tool is represented as a disc with a radius r. When a cut is requested, it is added to a queue to be processed after the current simulation timestep. Once the timestep gets completed and before the actor updates, all pending requests are dequeued, and the framework checks for intersections between the cutting disc and the volume constraints. If a volume constraint intersects the disc, it is disabled, and the simulation mesh is flagged as updated, prompting the actor to rebuild the render mesh.

Additionally, there is an option to immediately remove intersected distance constraints alongside the volume constraints. Activating this option eliminates the need for the slower residual distance constraint cleaning process typically performed by the dynamic softbody actor, improving performance in cutting-heavy simulations.

Cutting by Splitting Volume Constraints
Rather than removing entire tetrahedra, XR-PBD provides an algorithm to split them into smaller parts. When intersections are detected, new particles are generated at the points of intersection. Intersected distance constraints are divided in two and reconnected to these new particles. Similarly, volume constraints are split and re-tetrahedralized according to one of four unique cases, as illustrated in Figure 5.24.

Due to the nature of the algorithm, when the intersection of the disc lies near already existing particles, the retetrahedralization creates sliver tetrahedra, that cause artifacts, due to numerical inprecisions in the solver. To alleviate this issue, a second pass is performed to remove any generated tetrahedron that consists of, very short edges or has an insufficiently small volume.

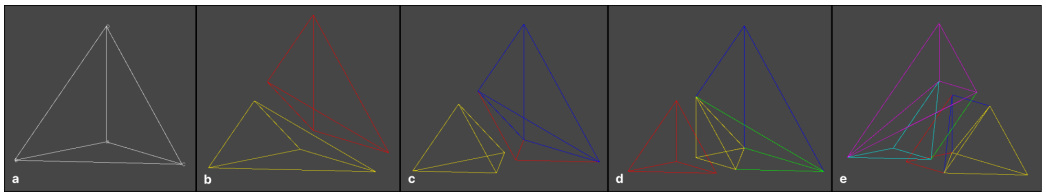

Figure 5.24: The 4 unique cutting cases of a tetrahedron based on number of intersected edges. a) The original tetrahedron. b) One edge intersection. c) Two edge intersections. d) Three edge intersections. e) Four edge intersections.

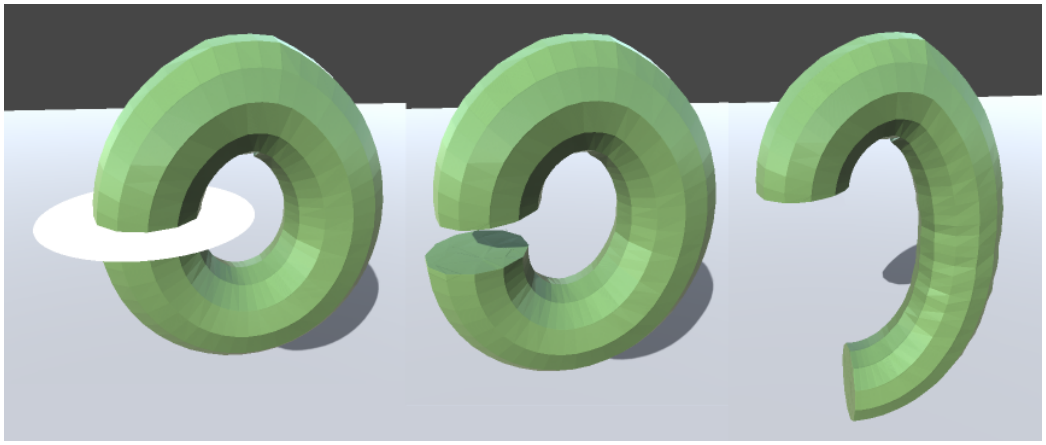

Figure 5.25: Cutting a hanging torus by splitting using a cutting disc (white).

5.7.4 Cloth

In XR-PBD, cloth simulation is achieved using Distance Constraints. Particles are placed at the vertices of the cloth mesh, representing the physical points of the simulated object. Distance Constraints are then applied along the edges of the mesh's triangles, creating a network of interconnected particles. This structure allows the cloth to respond dynamically to external forces, collisions, and interactions, while preserving the relative distances between connected particles.

Traditional simulation frameworks often include specialized algorithms to address interpenetration between cloth segments. In contrast, XR-PBD avoids interpenetration by leveraging the existing collision response system. This is achieved by carefully adjusting the particle sizes to ensure that the particles fully cover the cloth surface, preventing segments from passing through one another.

5.7.4.1 Rendering

The rendering of the cloth simulation mesh is tightly coupled with the underlying particle-based simulation. To visually represent the cloth, the vertices of the render mesh are bound to the particles in the simulation mesh. This binding process

is performed during initialization, where each vertex of the render mesh is associated with its nearest particle. At runtime, after each simulation time step, the positions of the render mesh vertices are updated to match the positions of their corresponding particles.

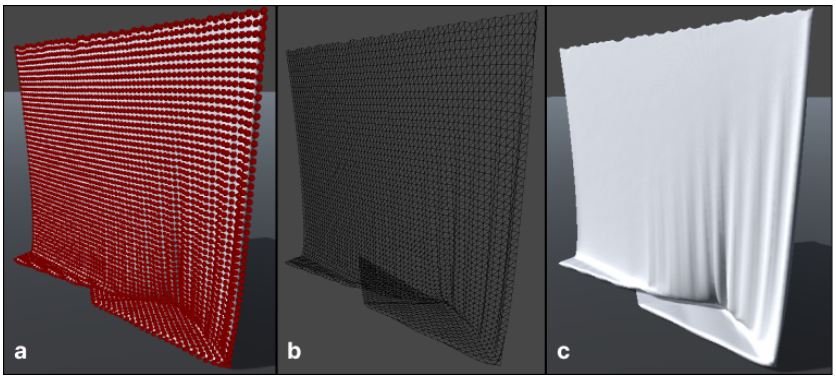

Figure 5.26: Cloth simulation. a) The cloth particles. b) The high resolution deformed render mesh. c) The final shaded cloth.

5.7.5 Rope

Ropes are simulated using only distance constraints. Particles are placed along a line at specified intervals, which are then connected with distance constraints to form the simulation mesh used for ropes.

5.7.5.1 Rendering

To render ropes, we developed a custom renderer that dynamically generates cylindrical geometries at runtime. Following each simulation time step, a Catmull-Rom spline is computed based on the positions of the particles, providing a smooth curve that approximates the shape of the low-resolution simulation mesh. This curve is sampled at regular intervals to generate a sequence of points. For each sampled point, a circle is created with the defined number of segments, oriented such that its normal vector aligns with the direction of the next point in the sequence. These circles are then connected to form the rope's outer surface. Additionally, optional end caps can be generated by connecting the vertices of the circles at the end points, to create a disc, resulting in a seamless and visually complete rope geometry.

To texture the dynamically generated rope tube mesh, we implemented a UV mapping system that unravels the mesh into a 2D plane and assigns UV coordinates to its vertices. This process ensures that textures are applied seamlessly across the surface of the rope. The UV mapping is performed as follows:

1. **Longitudinal Mapping**: The length of the rope is mapped onto the horizontal (U) axis of the texture. The U-coordinate of each vertex is calculated

based on its relative position along the spline curve, normalized by the total length of the rope. This ensures that the texture scales proportionally with the rope's length.

2. **Radial Mapping**: The circular cross-section of the rope is mapped onto the vertical (V) axis of the texture. For each vertex in a circle, the V-coordinate is determined by its angular position around the circle, with values distributed evenly from 0 to 1.

3. **Seam Handling**: A consistent seam is maintained along the length of the rope by ensuring that the UV coordinates for the first and last vertex of each circle align correctly, preventing visible texture mismatches or stretching.

By unwrapping the rope tube in this manner, the UV coordinates define a coherent mapping from the 3D surface of the mesh to the 2D texture plane. This approach allows the rope to be textured smoothly, regardless of its shape or curvature.

5.7.5.2 Stiff Rods

Traditional methods for simulating stiff rods based on Extended Position-Based Dynamics (XPBD) [7] [31] typically involve incorporating rotational information into the particles representing the rod. This is achieved either by directly adding rotation data to the particles or by introducing additional segments and virtual particles to store this rotational information. While these techniques enable highly realistic simulations capable of capturing effects such as torsion, they often come at the cost of significant computational overhead, making them unsuitable for complex real-time simulation scenarios.

In XR-PBD, stiff rods are simulated by leveraging a rope simulation enhanced with additional bending constraints to preserve the rod's original shape. Specifically, collinear constraints are applied every three particles along the rod. These constraints maintain the rod's structural integrity by restricting the angles formed by adjacent sections, relying solely on particle positions and avoiding the need for added rotational data. This approach strikes a balance between computational efficiency and visual fidelity, making it well-suited for real-time applications.

To provide flexibility, our stiff rod implementation is integrated in the rope actor mentioned earlier, only exposing an additional collinear constraint compliance parameter to the developer. By adjusting this parameter, developers can seamlessly transition between rod-like behavior, achieved by lowering the collinear constraint compliance, and more flexible rope-like behavior, achieved by increasing the compliance to reduce the influence of the bending constraints.

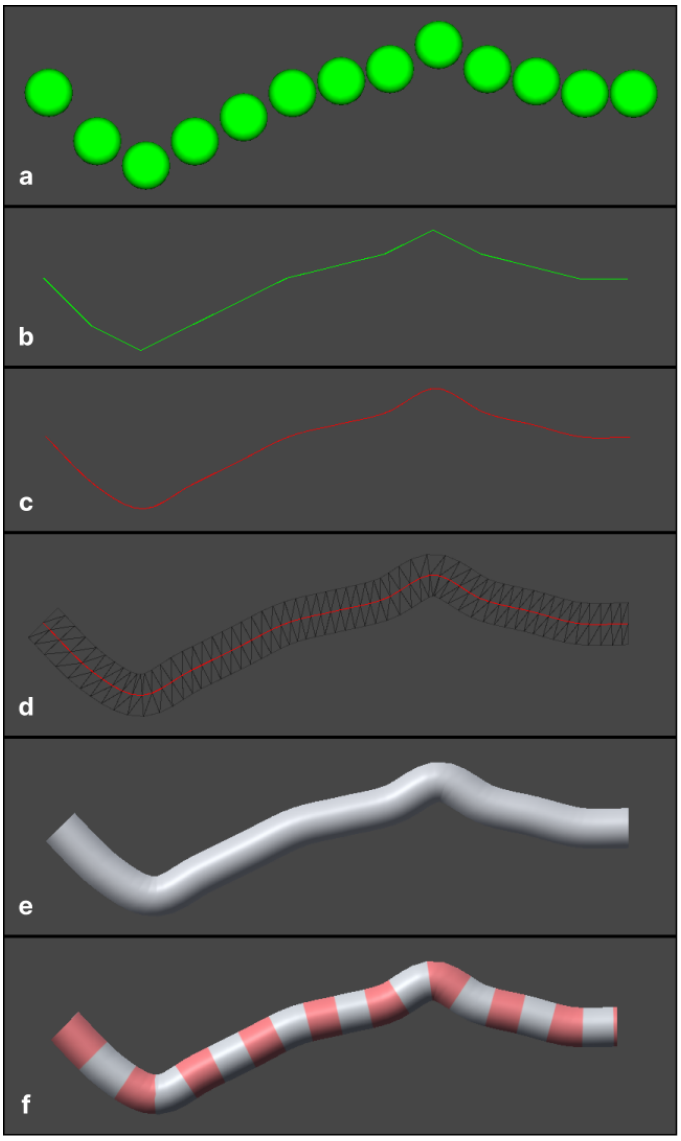

Figure 5.27: Rope/Rod rendering. a) The particles of the simulation mesh of a deformed rope/rod. b) The line segments formed by the particles. c) The smooth Catmull-Rom spline generated from the particles. d) The triangulation of the generated mesh. e) The shaded generated mesh. f) The final UV mapped shaded mesh.

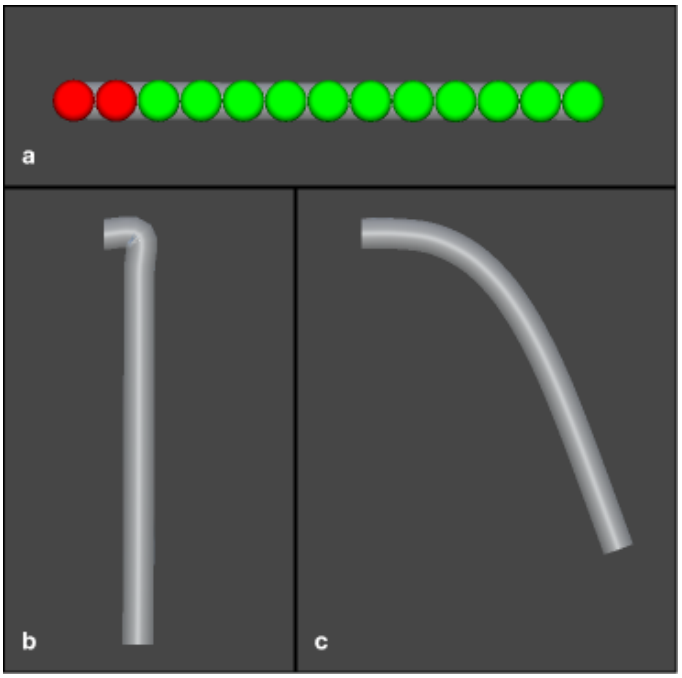

Figure 5.28: Rope vs Rod. a) The particles of the simulation mesh of the rope/rod. The red particles are pinned in place, while the green are free to move. b) Simulation with a high bending compliance, mimicking a rope. c) Simulation with a low bending compliance mimicking a stiff rod.

5.7.5.3 Cutting and Tearing

To enable tearing caused by extreme stretching or bending, the forces exerted on the particles of the simulation mesh by each constraint are monitored. If these forces exceed a developer-defined tear threshold, the corresponding Distance Constraint is disabled. To maintain simulation consistency and prevent artifacts, any associated collinear constraints are also disabled, and vice versa, ensuring a consistent and realistic deformation behavior.

In addition to force-based tearing, we also provide a cutting mechanism. When a cut is initiated, a cutting disc, with a given radius is employed to detect intersections with the constraints. Any constraints that intersect with the cutting disc are subsequently disabled.

Whenever a constraint is disabled, whether due to tearing or cutting, the simulation mesh is flagged as updated. This update is detected by the actor, which triggers a mesh rebuild. The rebuilding process involves applying the rendering algorithm described earlier separately to each newly formed rope or rod section created by the cut. This ensures that the visual representation of the mesh remains consistent with the underlying simulation.

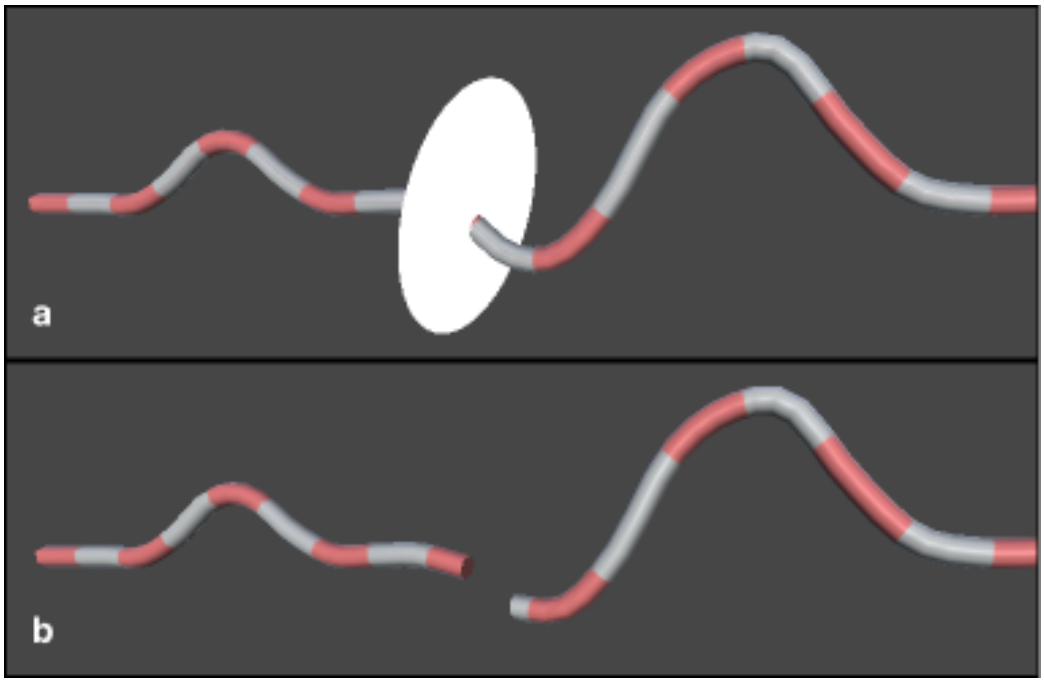

Figure 5.29: Rope cutting. a) The original rope to be cut with the white disc. b) The rope after cutting.

5.8 Detectors

While XR-PBD includes the most commonly needed detector types, we also provide a flexible base class that developers can extend. This base class grants easy access to the simulation mesh data, allowing developers to implement custom detector types tailored to their specific requirements. By combining out-of-the-box functionality with extensibility, our detector system streamlines real-time mesh monitoring for developers of varying expertise levels.

5.8.1 Particle Disconnection Detector

For the majority of the physics bodies described previously, particles are interconnected through a network of Distance Constraints. The Particle Disconnection Detector monitors these pairs of particles and triggers events when the Distance Constraints linking them are disabled. To achieve this, the detector constructs a graph representation of the simulation mesh, where particles are represented as nodes and Distance Constraints as edges.

When the simulation mesh is flagged as updated, the detector traverses the graph starting from one of the particles in the monitored pair, marking each visited node. After the traversal is complete, if the other particle in the pair remains unmarked, it indicates that the particles are no longer connected through the

simulation mesh, and a disconnection event is triggered.

For complex simulation meshes containing multiple sub-meshes, developers may prefer to restrict connection checks to a specific sub-mesh. In such cases, the graph is constructed using only the particles and Distance Constraints belonging to the specified sub-mesh, ensuring that the detector operates efficiently and focuses on the relevant subset of the mesh.

Instead of observing the simulation mesh for updates in order to do particle connection checks, developers can trigger the process manually at any time. In that case, the system can utilize the previously created graph instead of a creating a new one, thereby reducing computational overhead.

5.8.2 Distance Constraint Destruction Detector

In simulation meshes where distance constraints are used, someone may want to monitor specific regions of the mesh for destruction. Distance Constraint Destruction Detector monitors specific groups of distance constraints and triggers events when any of the distance constraints in that group gets disabled. To avoid constant computational overhead, checks for disabled distance constraints are performed in parallel and only when the simulation mesh is flagged as updated.

5.8.3 Volume Constraint Destruction Detector

The Volume Constraint Destruction Detector operates similarly to the Distance Constraint Destruction Detector. It monitors groups of volume constraints and triggers events whenever any constraint within the group is disabled. To optimize performance, these checks are executed in parallel and only during simulation mesh updates.

Defining groups of volume constraints manually by specifying indices can be time-consuming and unintuitive. To enhance usability, we provide a tool that allows developers to select volume constraints directly by clicking on them within the scene. When the developer clicks, a raycast is performed from the camera, identifying intersections with the tetrahedra defined by the volume constraints. The intersected constraints are then sorted based on their depth relative to the camera, and the closest one is selected. The index of this constraint is either added to or removed from the specified group, streamlining the process of group creation and modification.

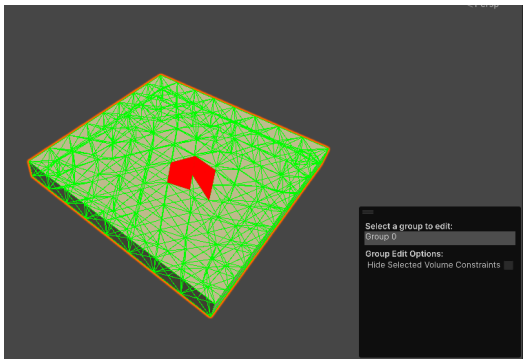

Figure 5.30: The Volume Constraint Destruction Detector editor tool used to select volume constraints by clicking on them.

5.8.4 Puncture Detector

The puncture detector is designed to work in conjunction with a Softbody Actor configured for piercing interactions. It monitors a specified group of triangles for punctures and triggers an event whenever an insertion constraint is detected for any of the monitored triangles after a simulation timestep.

Manually defining the group of triangles requires identifying their indices and adding them to a list, which can be time-consuming and error-prone. To improve usability, we provide an intuitive Editor Tool that allows developers to select triangles by simply clicking on them in the scene. When the developer clicks, a raycast is performed from the camera into the world, gathering all intersected triangles. The index of the triangle closest to the camera is then identified and added to the monitored group, streamlining the setup process.

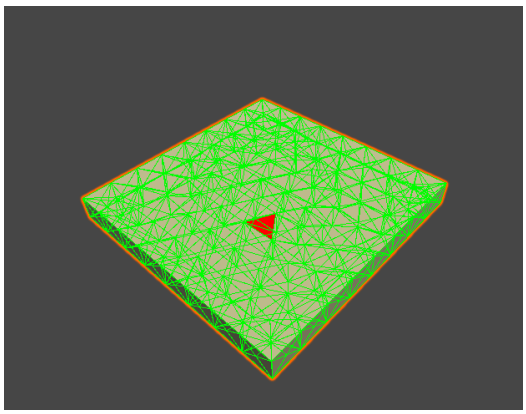

Figure 5.31: The Volume Puncture Detector editor tool used to select exterior surface triangles by clicking on them.

5.8.5 Floor Collision Detector

Detecting when an object is dropped onto the floor can be valuable in many scenarios. To address this, the Floor Collision Detector is used. This component monitors the particles of a specified simulation mesh and triggers an OnFloorCollisionEnter event whenever any particle collides with the floor. Additionally, when the collision stops, an OnFloorCollisionExit event is triggered, providing an easy way to track floor interactions.

5.9 Additional Optimization Techniques

To achieve optimal performance, particularly on low-end head-mounted displays (HMDs), we employed state-of-the-art additional runtime optimization techniques, including Unity's Job System, Burst Compiler and an SoA approach for data organization.

The Unity Job System, a multithreading framework, allowed us to design highly parallelized code in a structured and efficient manner. By enabling tasks to execute independently across multiple CPU cores, it maximizes the performance potential of modern hardware. This framework is built around the concept of jobs, self-contained units of work that can be scheduled and executed asynchronously. Adopting a job-based architecture allowed us to offload computationally intensive tasks, such as physics calculations, from the main thread, significantly reducing bottlenecks and improving overall application performance.

Furthermore, the Burst Compiler was utilized to complement the Job System, providing additional layers of optimization. The Burst Compiler is a high-performance compiler integrated into Unity, designed to translate managed C# code into highly optimized assembly. Burst enhances performance through advanced optimizations, such as, use of SIMD instructions (Single instruction,multiple data), vectorization, inlining, and efficient memory layout adjustments, tailored to modern processor architectures. By eliminating unnecessary abstraction layers and Just In Time(JIT) compilation, it minimizes runtime overhead and ensures deterministic and predictable execution, making it especially effective for computationally intensive operations.

Finally, in our implementation, we adopted a Structure of Arrays (SoA) approach instead of the traditional Array of Structures (AoS) for data organization. This decision was made to align with the optimization goals of the Burst Compiler and Unity's Job System. SoA improves data locality and memory access patterns, which are critical for achieving high performance on modern CPUs. By organizing data in contiguous memory blocks for each property, SoA allows vectorized operations to process multiple elements simultaneously and reduce cache misses, leveraging the full potential of the processor. However, for simplicity and consistency, we refer to these data structures as "structs" in the previous sections, even though their underlying implementation follows the SoA paradigm.

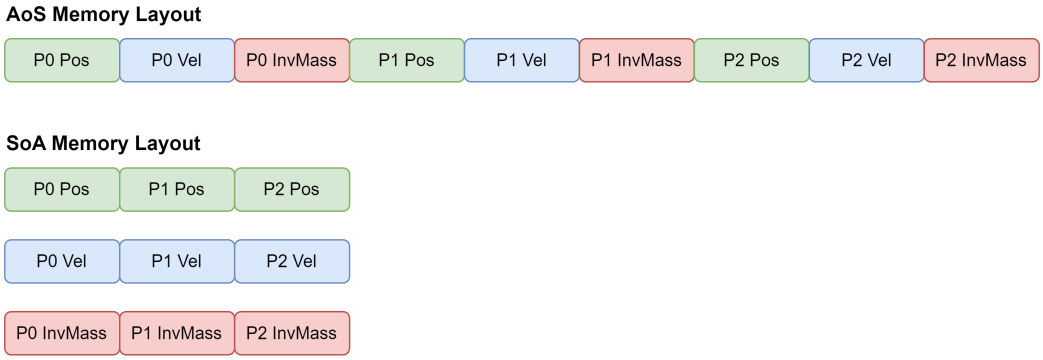

Figure 5.32: Array of Structures (AoS) vs Structure of Arrays (SoA) memory layout.

Chapter 6

Results & Performance Metrics

To showcase the capabilities of our framework and evaluate performance, we designed a series of scenarios that include both realistic use cases and complex stress-test environments. While most scenes focus on simulating a single object type, some include interactions between multiple object categories. Performance metrics were measured both on a laptop with an Intel(R) Core(TM) i9-14900HX @ 2.20 GHz, NVIDIA GeForce RTX 4060 Laptop GPU and 32.0GB RAM, also mentioned as "PC" in the following sections and a Meta Quest 2 mobile HMD. Additionally, to further analyze performance and showcase simulations running on mobile HMDs, we developed interactive VR-exclusive scenarios that require real-time user input. A showcase video of the simulation scenes described in the following sections, can be found at: `https://youtu.be/iYr_sj3qjVs`

6.1 Rigidbody

To showcase rigidbodies and assess their performance, we instantiated 40 rigidbodies at random positions, dropping them from the ceiling (Figure 6.1). These rigidbodies comprised a total of 8850 particles and an equal number of shape-matching constraints. The simulation was configured with two solver substeps, two iterations for collision handling, and one iterations for shape-matching constraints. Rendering was managed natively by Unity, incurring no additional performance overhead. The simulation required 8.27 ms per time step on a PC and 17.05 ms on a Meta Quest 2 HMD.

Unlike traditional rigidbody-focused simulation engines, simulating each rigidbody with calculations only for the center of mass, XR-PBD simulates each particle individually. This causes XR-PBD to not perform as efficiently compared to these traditional rigidbody simulation frameworks, but allows us to have a lightweight collision detection pipeline and excel in soft-rigid coupling scenarios.

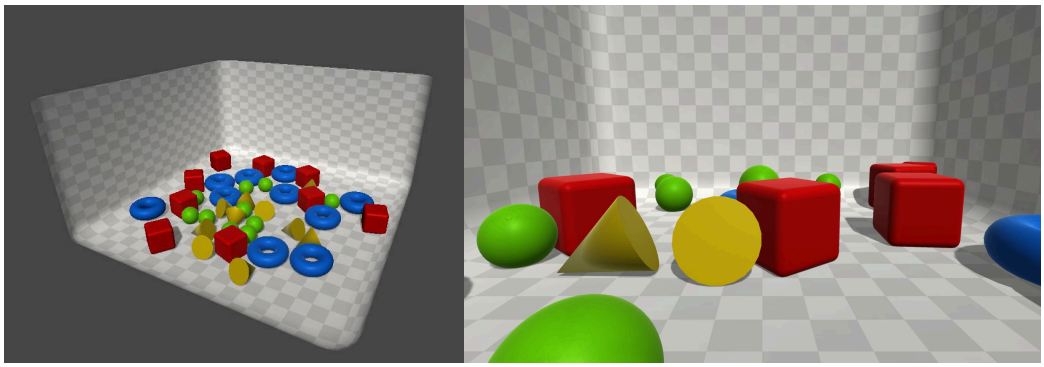

Figure 6.1: 50 rigidbodies falling from the sky.

6.2 Softbody

To showcase softbodies and assess their performance, we designed a scene featuring a soft jelly. To induce deformation and evaluate soft-rigid interactions, we introduced two rigidbody actors simulating sugar cubes falling on top of a jelly. The simulation includes 1815 particles, 9122 distance constraints, 5814 volume constraints, and 18 shape-matching constraints. The solver operates with 2 substeps, utilizing 10 iterations for collision handling, 4 iterations for distance and volume constraints, and 1 iteration for shape-matching constraints. The render mesh of the jelly consists of 1646 vertices and 1944 triangles, while the render mesh of each sugar cube consists of 24 vertices and 12 triangles. Each solver loop needs 18.13 ms on a Meta Quest 2 HMD and 13.13ms on PC.

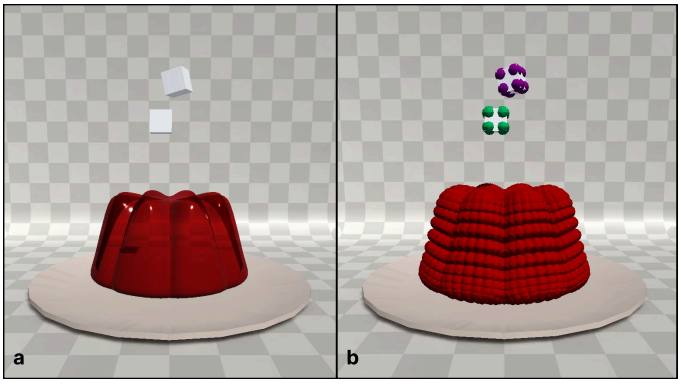

Figure 6.2: Particle configuration for the simulation of sugar cubes falling on top of a jelly.

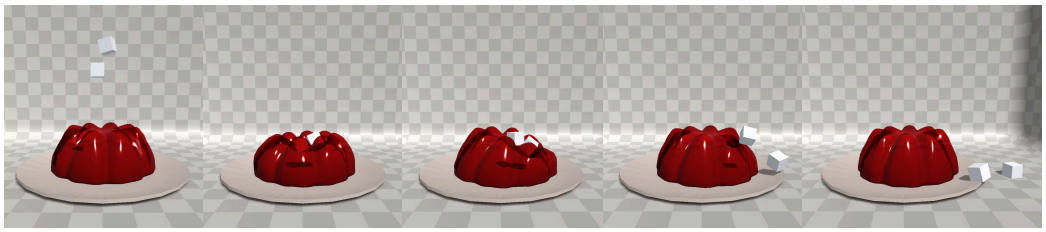

Figure 6.3: Timelapse of the simulation of sugar cubes falling on top of a jelly.

6.2.1 Piercing Performance

To assess piercing performance, we created an additional scene (Figure 6.4). In this scenario, a soft, thin sheet is fixed on three sides while a hook pierces and stretches it. The simulation mesh of the sheet consists of 484 particles, 2281 distance constraints, and 1316 volume constraints, while its render mesh comprises 833 vertices and 1352 triangles. The hook simulation includes 9 particles and 9 shape-matching constraints.

The simulation was configured with 2 solver substeps, 10 iterations for collision resolution, 10 iterations for distance constraints, 4 for volume constraints, and 1 for shape matching. Additionally, insertion constraints were resolved using 4 iterations. Each solver loop and rendering update required 3.8 ms on a laptop and 9.7 ms on a Meta Quest 2 portable HMD, as detailed in Figures 6.1 and 6.2.

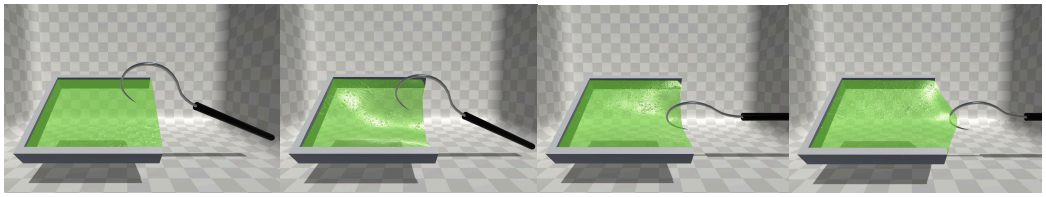

Figure 6.4: Timelapse of a hook piercing and pulling a thin deformable sheet.

6.3 Dynamic Softbody

To assess the performance of dynamic softbodies and showcase their capabilities, we used the jelly as in the previous, regular softbody evaluation scenario. However, instead of rendering a pre-existing embedded mesh, the mesh was dynamically generated at runtime by the dynamic softbody actor, having the smooth shading option enabled. Notably, no visual difference was observed between the dynamically generated mesh and the pre-made mesh used in the original softbody evaluation.

6.3.1 Tearing

In the previously described scene, a knife was introduced with particles positioned along its blade. Tearing was enabled in the jelly's simulation mesh, with the tear force threshold set to 1e+09 in the physics material. To be able to inspect the incision after the tear, some particles from each piece were attached to two forks placed within the scene. When the tearing process is complete the two forks, move apart and reveal the incision area. (see Figure 6.6).

The tearing process itself incurred no additional computational cost in terms of simulation performance. The only overhead came from rebuilding the render mesh when the simulation mesh was modified. In general, the frame time remained around 3.3 ms on PC and 15 ms on the Meta Quest 2 when no mesh rebuild was required. When a mesh rebuild was triggered, an additional overhead of 0.03 ms and 0.1 ms was added on PC and Quest 2 accordingly.

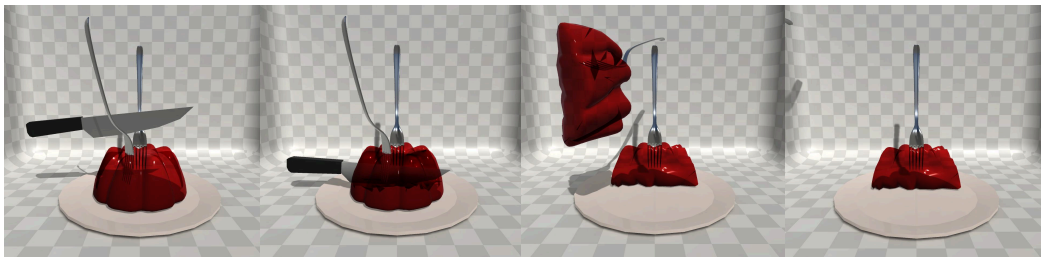

Figure 6.5: Timelapse of a jelly being torn, due to the forces applied to it from a knife. The jelly pieces are separated to reveal the torn area.

6.3.2 Cutting

To assess our framework's performance in handling softbody cutting, we designed another dedicated test scenario. In this scene, a soft ball, suspended from a rope, is cut by a cleaver while swinging. The ball's simulation consists of 225 particles, 1178 distance constraints, and 794 volume constraints, while the automatically generated render mesh of it, contains 162 vertices and 320 triangles. The rope's simulation mesh includes 19 particles, 18 shape matching constraints, and 17 collinear constraints.

Performance measurements were taken both during regular simulation frames and during frames where a cut was performed. Both cutting methods, cutting by splitting volume constraints and cutting by removing them completely, were measured to compare their computational overhead.

Cutting by Removing Volume Constraints

For this scenario, to detect and disable volume and distance constraints an additional time of 9 ms was used on PC. Each solver loop required 3.22 ms, while

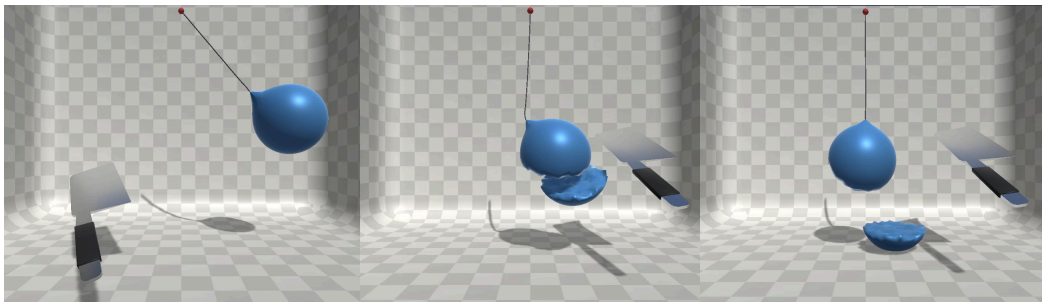

Figure 6.6: Timelapse of a ball hanging from a rope being cut by a cleaver.

updating the render mesh took 0.1 ms. After each cut, an additional 0.03 ms was needed to reconstruct the render mesh. Enabling the mesh cleanup options introduced a further 0.1 ms overhead for removing volume constraints with one or fewer neighbors, while residual distance constraint removal added 0.03 ms to the frame time.

Cutting by Splitting Volume Constraints

Unlike the previous method, cutting by splitting volume constraints introduces additional computational overhead. This is due to the complex calculations required to generate new tetrahedra and the memory allocations needed to accommodate the newly created constraints and particles within the physics world.

Initially, after the first cut, the simulation time remains 3.22 ms per solver loop, but increases slightly after each cut due to the growing number of constraints. The first cut takes approximately 42.2 ms, while subsequent cuts require around 9.8 ms. This discrepancy arises from the way internal buffers are resized, doubling in size with each cut. Consequently, the first few cuts incur higher costs due to memory allocation, whereas later cuts may avoid additional allocations if sufficient memory has already been reserved.

To mitigate this overhead, developers can mark the simulation mesh as dynamic, ensuring that a larger than regular memory pool is preallocated before the simulation starts, thereby reducing the need for costly runtime memory allocations.

Additional performance measurements were taken on PC in the cutting scene shown in Figure 7.3. Resulting Simulation Update FPS and Render Update FPS are shown in the following graphs in Figure 6.7 and Figure 6.8 respectively, both for our framework, XR-PBD and SOFA.

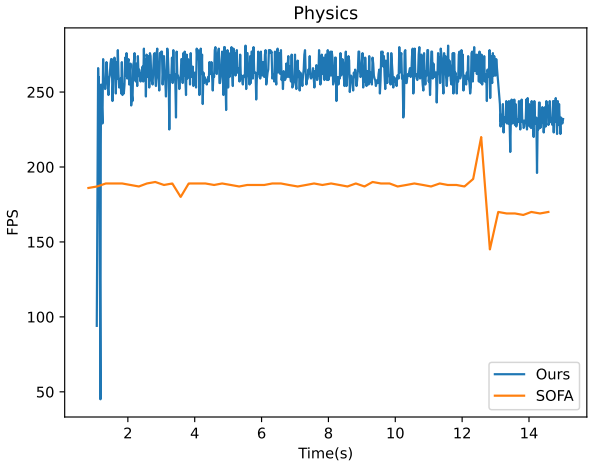

Figure 6.7: Physics Update FPS in XR-PBD, compared to SOFA. Initial spikes in performance in our framework are attributed to initialization and memory allocations happening when a body is added to the PhysicsWorld.

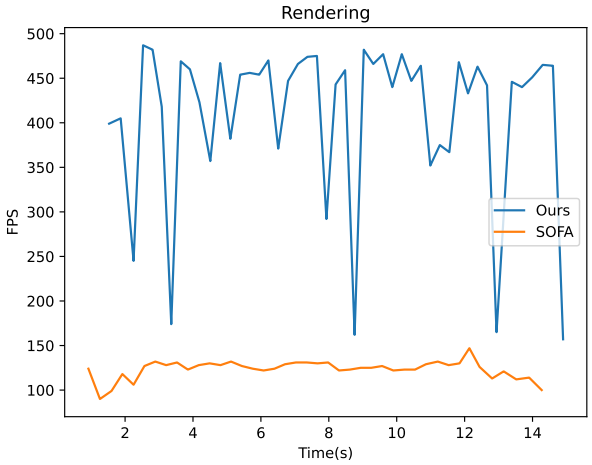

Figure 6.8: Render Update FPS in XR-PBD, compared to SOFA. Spikes in measurements in our framework originate from Garbage Collection tasks and Job scheduling logic executed in main thread before a solver loop starts.

XR-PBD is 39% faster than SOFA in the aforementioned cut simulation case as far as physics simulation is concerned, and 230% faster in rendering.

Frame drops in the Physics Update FPS of XR-PBD, at the beginning of the simulation, are attributed to the initial resizing of the physics world data performed in order for the Softbody to be added to it.

Frame drops in the Render Update FPS of XR-PBD during the simulation, originate from Garbage Collection tasks and Job scheduling logic executed in main thread before a solver loop starts.

6.4 Cloth

Cloth performance was assessed through a flag simulation scenario (Figure 6.9), where a flag is attached to a flagpole and subjected to wind forces. The simulation consists of 1089 particles and 3136 distance constraints. The solver operates with two substeps, utilizing 1 iteration for collisions and 20 iterations for distance constraint resolution. To prevent interpenetration, cloth self-collision is enabled. Wind effects are simulated by applying random forces to the particles in the direction of the wind. The render mesh of the cloth is a subdivided quad consisting of 1089 vertices and 2048 triangles. This simulation scene needs 3.92 ms on a Meta Quest 2 HMD and 11 ms on PC for each solver loop.

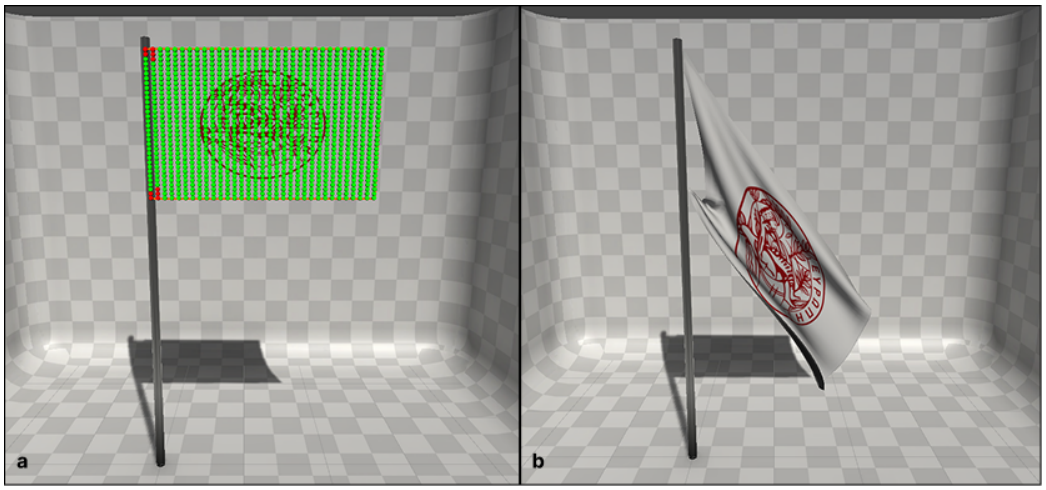

Figure 6.9: Flag simulation. a) The particles of the simulation. b) The simulated flag.

6.5 Rope/Rod

To evaluate performance and stress test XR-PBD, we created a scene featuring 45 falling "noodles," simulated as semi-stiff rods. The simulation consists of 912 particles, 864 distance constraints, and 816 collinear constraints, with 2 solver substeps, 4 iterations for distance constraints, and 1 iteration for collision and collinear constraints. As demonstrated in Figures 6.1 and 6.2, the scene runs smoothly in real time both on high-performance devices and low-performance mobile HMDs, requiring only 1.05 ms and 2.8 ms for each solver iteration respectively.

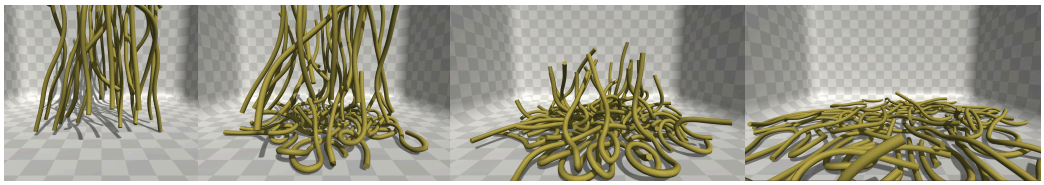

Figure 6.10: Timelapse of falling noodles.

6.5.1 Cutting

To further assess the behavior of the rope actor in cases were multiple collisions are present, and evaluate the cutting performance, a new scene was used. In this scene three cables each consisting of 50 particles, 49 distance constraints and 48 collinear constraints are twisted. The simulation is configured with 4 solver substeps, 10 iterations for collision handling, 4 iterations for distance constraint resolution and 1 iteration for the collinear constraints. Each solver loop consumes 3.01 ms on a Meta Quest 2 and 3.47 ms on PC.

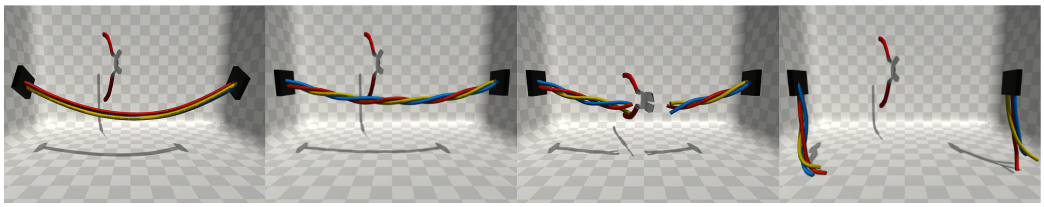

Figure 6.11: Timelapse of cable twisting and cutting.

Measurements taken from the above scenes, shown also in Table 6.1 and Table 6.2, prove that XR-PBD is capable to simulate complex scenarios in real-time not only in high performance devices, but also in low-performance portable HMDs.

To evaluate deeper which part of our simulation loop consumes most time we performed measurements in each individual step of the solver loop. Results showed that the majority of the time needed for one simulation loop is consumed in rebuilding the spatial hash grid and collecting particle-particle contacts. Less time is needed for the constraint solving and even less is needed for updating the render mesh.

Table 6.1: Average time needed for a time step on a portable Meta Quest 2 HMD.

Scene	**Simulation Loop** (ms)	**Render Mesh Update** (ms)	**Total** (ms)
Rigidbody Rain	17.05	-	17.05
Jelly	18.09	0.05	18.13
Hook Pulling	9.64	0.05	9.69
Jelly Tearing	14.98	0.04	15.02
Ball Cutting	6.25	0.05	6.3
Flag	10.05	0.05	11
Falling Noodles	2.66	0.14	2.8
Cable Twist and Cut	2.76	0.25	3.01

Table 6.2: Average time needed for a time step on PC.

Scene	**Simulation Loop** (ms)	**Render Mesh Update** (ms)	**Total** (ms)
Rigidbody Rain	8.27	-	8.27
Jelly	13.09	0.04	13.13
Hook Pulling	3.75	0.05	3.8
Jelly Tearing	3.24	0.04	3.28
Ball Cutting	3.22	0.03	3.25
Flag	3.87	0.05	3.92
Falling Noodles	1	0.05	1.05
Cable Twist and Cut	3.38	0.09	3.47

6.6 Untethered VR Use cases

To prove that XR-PBD is capable of simulating real-world scenarios with user interactions in mobile XR HMDs, we have created some additional scenarios that run in realtime in mobile HMDs.

In Figure 6.12 the VR user tries to remove a puzzle-shaped piece from the upper layer of a two layer sphere. This simulation mesh consists of 2 submeshes with 3251 particles, 17408 distance and 11837 volume constraints. Scene performs as expected in a Meta Quest 2 portable HMD, achieving a steady framerate of 70 FPS.

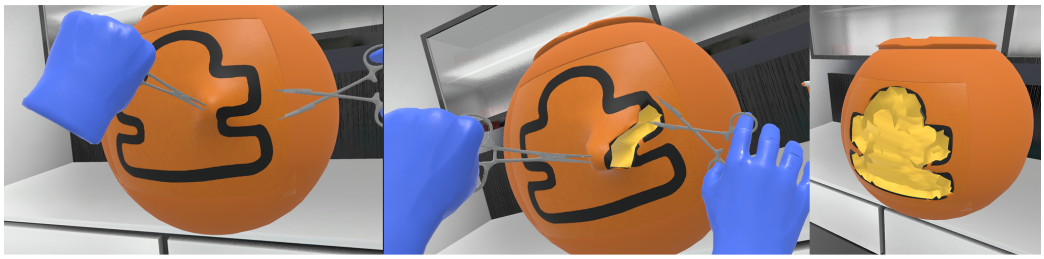

Figure 6.12: VR simulation of a user cutting and removing a puzzle shaped surface piece from a soft sphere.

In Figure 6.13 the VR user cuts cables attached between two junction boxes. This simulation consists of 148 Particles, 141 Distance and 138 Collinear Constraints. It is simulated on a Meta Quest 2 portable HMD with a stable framerate of 70 FPS.

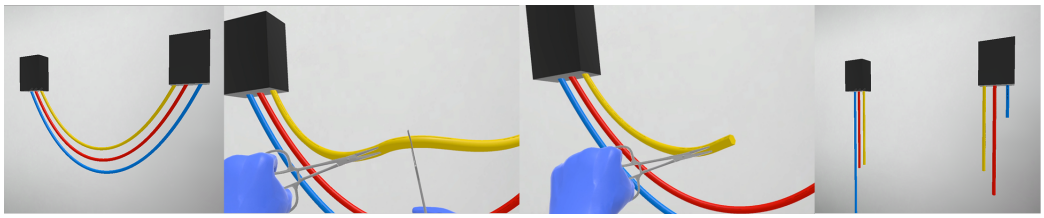

Figure 6.13: VR Simulation of a user cutting the cables connecting two junction boxes.

In Figure 6.14 the VR user pulls apart two softbody thin sheets, glued together. This simulation consists of a single simulation mesh with 3 sub-meshes, one for each visible material. In total, 992 particles, 5777 distance constraints and 4300 volume constraints were used. The simulation is able to run on a Meta Quest 2 portable HMD with a stable framerate of 70 FPS.

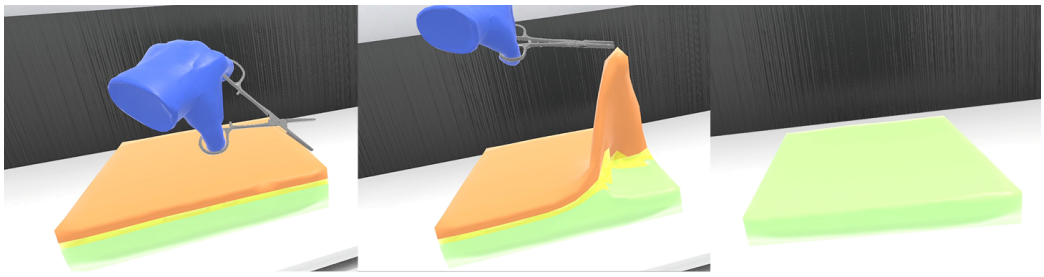

Figure 6.14: VR Simulation of a user pulling apart 2 glued sotftbody thin sheets.

Chapter 7

Evaluation

To assess the physical accuracy of XR-PBD, we measured and plotted the constraint error across different solver substep counts. Additionally, we evaluated the precision of our cutting algorithms by comparing our results to industry-standard solutions, including both offline and real-time desktop implementations. Lastly, to analyze performance, we conducted a series of test scenarios with varying complexity on both desktop systems and standalone mobile HMDs.

7.1 Constraints

To examine the effect of substep count on residual error at the end of a time step, we simulated a chain consisting of 50 particles connected by 49 distance constraints, hanging from a fixed point. The chain's length was recorded after each time step, and the deviation from its original length was used to compute the error percentage. As shown in Figure 7.1, increasing the substep count leads to an exponential decrease of the residual error.

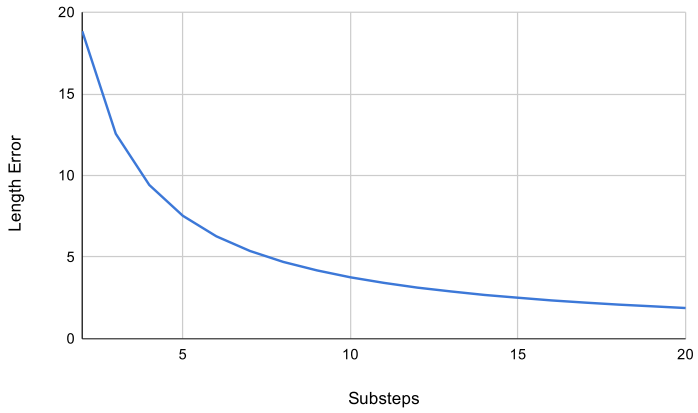

Figure 7.1: Solver substeps and Chain length error

7.2 Cutting Evaluation & Comparison with SOFA

To assess the accuracy of our cutting algorithms applied to dynamic softbody actors, we conducted experiments using an icosphere simulation mesh consisting of 1317 particles, 7952 distance constraints, and 5996 volume constraints. A predefined cut plane was used to perform cuts using both methods: removing entire tetrahedra and splitting them in half. As a ground truth reference, the same cut was executed in Blender[10], a widely used 3D modeling software. The resulting lower halves of the meshes, were then compared in Meshlab[6], using the Hausdorff distance metric, to quantify the differences between them.

To visualize the error distribution, the Hausdorff distance for each point was assigned a color, highlighting discrepancies in the mesh. The ground truth model and the corresponding results are presented in Figure 7.2.

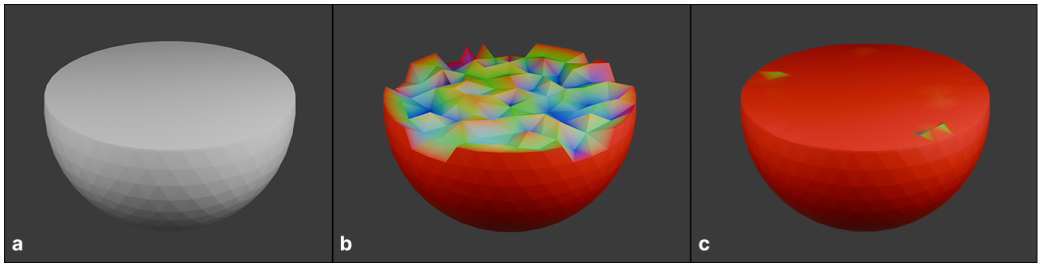

Figure 7.2: Visualization of Cut error. a) The ground truth. b) Cut by removing. c) Cut by splitting (after 2nd pass).

The results indicate that cutting by removing entire tetrahedra fails to preserve fine details in the cut region, leading to a rough and uneven surface. Reducing the tetrahedron size or applying smooth shading can mitigate these artifacts and improve the cut accuracy.

On the contrary, cutting by splitting tetrahedra yields significantly more accurate results, producing a clean and flat cut surface, similar to the ground truth. Although minor artifacts may remain, as shown by the Hausdorff distance visualization, they are imperceptible in most cases. These artifacts arise when sliver tetrahedra, generated after a cut, are removed in a second pass, to simplify the simulation and prevent numerical instability.

To further evaluate our cutting interactions in comparison to established physics frameworks, we conducted a direct comparison with an identical simulation in SOFA. The test scene involved a softbody object suspended in mid-air, being cut by a blade. For SOFA, we utilized its default cutting implementation, while in XR-PBD, we applied the Cutting by Splitting Volume Constraints method. The results of this comparison are presented in Figure 7.3, where the left side illustrates the SOFA simulation and the right side shows our approach. Our method demonstrated superior performance, achieving an average frame rate of 150 FPS compared to SOFA's 90 FPS. This difference in performance is attributed to the way

physics are simulated in each framework. Firstly, our method optimizes softbody collisions by utilizing a more efficient particle-particle collision pipeline, whereas SOFA employs a dynamically updated triangle collision shape, which, while more accurate, is computationally more expensive. Additionally, for the softbody SOFA implements a slower FEM co-rotational simulation, while we use a faster Parallel XPBD approach, which even though it is less accurate, it does not result in significant visual differences in the current simulation.

Additionally, our approach produced more precise cuts, particularly at the intersection boundaries, as highlighted in Figure 7.4, which provides a detailed view of the cut profiles. This enhancement arises from our approach to tetrahedra splitting. XR-PBD accurately processes tetrahedra with only one or two intersected edges, whereas SOFA assumes all intersections involve three edges, leading to incorrect re-tetrahedralization of boundary elements.

Both frameworks encounter challenges with normal calculation in the generated faces, due to the method used for computing smooth normals. Specifically, no smoothing groups or the angle between faces are considered in the process, which would otherwise introduce additional vertices at these points.

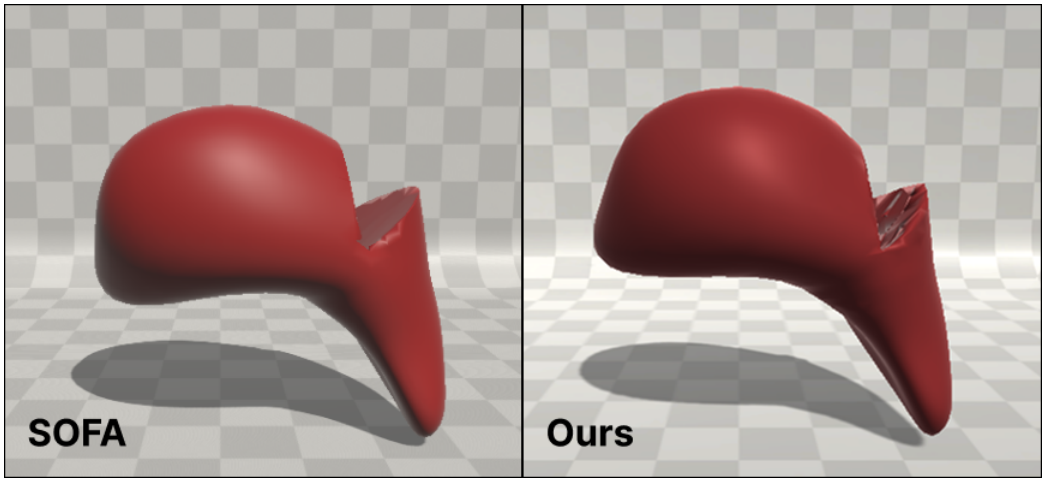

Figure 7.3: Cut in SOFA vs Ours.

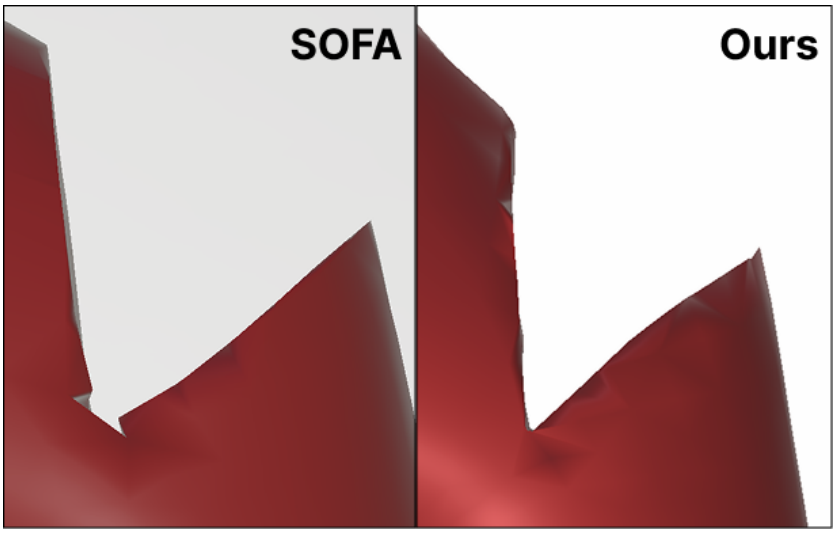

Figure 7.4: Cut in SOFA vs Ours close-up. Our method produces no visible artifacts in boundary elements, compared to SOFA

Chapter 8

Conclusion

In this thesis, we introduced XR-PBD, a robust and versatile real-time physics framework designed to enable advanced physics interactions such as cutting, tearing, and piercing on low-performance mobile HMDs. Our approach enhances the efficiency, accuracy, and interactivity of physics-based simulations in extended reality (XR) environments.

At the core of XR-PBD is an innovative XPBD-based solver. This solver builds upon a modified massively parallel Jacobi solver and introduces the concept of Successive Over-Relaxation (SOR) to improve convergence. By incorporating these enhancements, our solver achieves greater numerical stability and faster constraint resolution compared to traditional XPBD approaches. It leverages a broad range of constraint types to accurately simulate rigidbodies, softbodies, cloths, ropes, and stiff rods.

To achieve high performance, even on hardware with limited computational resources, XR-PBD integrates several optimization strategies. These include SIMD (Single Instruction, Multiple Data) acceleration and efficient memory management, reducing computational overhead while maintaining simulation stability. These optimizations allow XR-PBD to outperform other state-of-the-art physics engines in terms of both accuracy and speed, particularly in real-time deformable body interactions.

Physics objects and simulations are exposed to developers through an intuitive, object-oriented API, encapsulated in Actor components, which abstract away the complexity of the underlying data structures and physics computations. This design choice ensures ease of use, allowing developers to implement complex simulations without requiring deep physics knowledge or extensive coding.

Another key contribution of XR-PBD is the introduction of Insertion Constraints, a novel XPBD-based constraint type developed for real-time piercing interactions. This method ensures that piercing objects, such as needles or hooks, follow a physically accurate path while maintaining realistic frictional and resistive forces.

Additionally, XR-PBD expands upon existing cutting techniques by introducing two novel methods for real-time cuts on deformable models: Cutting by Removing Volume Constraints, a lightweight approach that instantly removes intersected volume constraints to simulate softbody cuts efficiently and Cutting by Splitting Volume Constraints, a more complex method that retetrahedralizes the mesh dynamically, preserving volume integrity and improving visual accuracy of cut surfaces.

To further enhance usability, XR-PBD incorporates event-driven interaction systems, called Detectors, that allow non-physics-programmers to easily define responses to simulation mesh changes using an editor-based GUI. Detector components continuously monitor the simulation in real time, triggering predefined events based on collisions or constraint and particle changes. This makes the framework highly adaptable to a vast amount of application categories, ranging from game development to education and training.

Finally, XR-PBD's performance and accuracy were extensively validated through real-world scenarios and comparative analysis with other renowned physics frameworks like SOFA and Unity PhysX. Benchmarks demonstrated that XR-PBD achieves better performance, even by 39% for the simulation and by 230% for rendering compared to SOFA, while maintaining visually similar results. The superior precision of our cutting techniques, particularly in intersection boundary handling and re-tetrahedralization, further underscores the advancements introduced by our approach.

Through these contributions, XR-PBD establishes a foundation for unconditionally stable, complex physics simulations with advanced user interactions even in low-performance mobile XR HMDs.

8.0.1 Future Work

Currently, relying solely on particle-particle collisions, while computationally efficient, is not well-suited for scenarios where the interacting objects differ significantly in scale. In the future, we aim to enhance our collision detection and response system by introducing additional collision shapes, such as primitives (e.g., cubes, spheres, and capsules). This will enable smaller rigid objects to interact more realistically with larger softbodies.

Furthermore, we plan to transition from using shape-matching constraints for rigid bodies to implementing a more detailed rigid body simulation, still based on Extended Position Based Dynamics (XPBD) [24]. This approach will support advanced joint constraints, significantly improving interactions with rigid tools and allowing for complex multi-part rigidbody simulation.

Additionally, we aim to upgrade our solver by incorporating additional GPU-based parallel computations with Compute Shaders, to be used as an alternative to our current solution. This enhancement will leverage the computational power of high-end desktop devices, where GPUs are often underutilized, to improve performance in demanding simulation scenarios and broaden the target device range.

Lastly, we would like to investigate how and if we could benefit from using Neural Fields[33] in our simulations.

Bibliography

[1] NVIDIA-Omniverse/PhysX, February 2025. original-date: 2022-10-04T09:07:32Z.

[2] Jan Bender. Impulse-based dynamic simulation in linear time. *Computer Animation and Virtual Worlds*, 18(4-5):225–233, 2007. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/cav.179.

[3] Jan Bender, Dieter Finkenzeller, and Alfred Schmitt. An impulse-based dynamic simulation system for VR applications. January 2005.

[4] Jan Bender, Matthias Müller, and Miles Macklin. A Survey on Position Based Dynamics, 2017. 2017.

[5] Jan Bender, Matthias Müller, Miguel Otaduy, Matthias Teschner, and Miles Macklin. A Survey on Position-Based Simulation Methods in Computer Graphics. *Computer Graphics Forum*, 33, April 2014.

[6] Paolo Cignoni, Claudio Rocchini, and Roberto Scopigno. Metro: measuring error on simplified surfaces. In *Computer Graphics Forum*, volume 17, pages 167–174. Blackwell Publishers, 1998.

[7] Crispin Deul, Tassilo Kugelstadt, Marcel Weiler, and Jan Bender. Direct Position-Based Solver for Stiff Rods. *Computer Graphics Forum*, 37(6):313–324, September 2018.

[8] Christer Ericson. *Real-Time Collision Detection*. CRC Press, 0 edition, December 2004.

[9] Francois Faure, Christian Duriez, Hervé Delingette, Jeremie Allard, Benjamin Gilles, Stéphanie Marchesseau, Hugo Talbot, Hadrien Courtecuisse, Guillaume Bousquet, Igor Peterlik, and Stéphane Cotin. SOFA: A Multi-Model Framework for Interactive Physical Simulation. volume 11. June 2012.

[10] Blender Foundation. Blender - Free and Open Source 3D Creation Software. `https://www.blender.org/`.

[11] M. Fratarcangeli and F. Pellacini. Scalable Partitioning for Parallel Position Based Dynamics. *Comput. Graph. Forum*, 34(2):405–413, May 2015.

[12] Thomas Jakobsen. Advanced character physics. *In Game Developers Conference Proceedings*, January 2001.

[13] Manos Kamarianakis, Nick Lydatakis, Antonis Protopsaltis, John Petropoulos, Michail Tamiolakis, Paul Zikas, and George Papagiannakis. "Deep Cut": An all-in-one Geometric Algorithm for Unconstrained Cut, Tear and Drill of Soft-bodies in Mobile VR, August 2021. arXiv:2108.05281 [cs].

[14] Manos Kamarianakis, Antonis Protopsaltis, Dimitris Angelis, Michail Tamiolakis, and George Papagiannakis. *Progressive Tearing and Cutting of Soft-bodies in High-performance Virtual Reality.* The Eurographics Association, 2022. ISSN: 1727-530X.

[15] Manos Kamarianakis, Antonis Protopsaltis, Michail Tamiolakis, and George Papagiannakis. Realistic soft-body tearing under 10ms in VR, May 2022. arXiv:2205.00914 [cs].

[16] Georgios Lampropoulos and Kinshuk. Virtual reality and gamification in education: a systematic review. *Educational technology research and development*, 72(3):1691–1785, June 2024.

[17] Tiantian Liu, Adam W. Bargteil, James F. O'Brien, and Ladislav Kavan. Fast simulation of mass-spring systems. *ACM Trans. Graph.*, 32(6):214:1–214:7, November 2013.

[18] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. XPBD: position-based simulation of compliant constrained dynamics. In *Proceedings of the 9th International Conference on Motion in Games*, pages 49–54, Burlingame California, October 2016. ACM.

[19] Miles Macklin, Matthias Müller, Nuttapong Chentanez, and Tae-Yong Kim. Unified particle physics for real-time applications. *ACM Trans. Graph.*, 33(4):153:1–153:12, July 2014.

[20] Brian Mirtich. Impulse-based Dynamic Simulation. April 2002.

[21] Jacob Moore, Harald Scheirich, Shreeraj Jadhav, Andinet Enquobahrie, Beatriz Paniagua, Andrew Wilson, Aaron Bray, Ganesh Sankaranarayanan, and Rachel B. Clipp. The interactive medical simulation toolkit (iMSTK): an open source platform for surgical simulation. *Frontiers in Virtual Reality*, 4, June 2023. Publisher: Frontiers.

[22] Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. Position based dynamics. *Journal of Visual Communication and Image Representation*, 18(2):109–118, April 2007.

[23] Matthias Müller, Miles Macklin, Nuttapong Chentanez, and Stefan Jeschke. Physically Based Shape Matching. *Computer Graphics Forum*, 41(8):1–7, December 2022.

[24] Matthias Müller, Miles Macklin, Nuttapong Chentanez, Stefan Jeschke, and Tae-Yong Kim. Detailed Rigid Body Simulation with Extended Position Based Dynamics. *Computer Graphics Forum*, 39(8):101–112, December 2020.

[25] NVIDIA. CUDA - Parallel Computing Platform. `https://developer.nvidia.com/cuda-zone`.

[26] NVIDIA. NVIDIA FleX. `https://developer.nvidia.com/flex`.

[27] Miguel Seabra, Francisco Fernandes, Daniel Lopes, and Joao Madeiras Pereira. *Position Based Rigid Body Simulation: A comparison of physics simulators for games.* May 2023.

[28] Eftychios Sifakis, Kevin Der, and Ronald Fedkiw. *Arbitrary cutting of deformable tetrahedralized objects.* January 2007. Journal Abbreviation: Proc. of Symp. on Computer Animation Pages: 80 Publication Title: Proc. of Symp. on Computer Animation.

[29] Virtual Method Studio. Obi - Unified Particle Physics for Unity 3D. `https://obi.virtualmethodstudio.com/`.

[30] Unity Technologies. Unity Engine - Real-Time 3D Development Platform & Editor. `https://unity.com/products/unity-engine`.

[31] Nobuyuki Umetani, Ryan Schmidt, and Jos Stam. Position-based elastic rods. *ACM SIGGRAPH 2014 Talks, SIGGRAPH 2014*, July 2014.

[32] Unity. Unity Physics. `https://docs.unity3d.com/Packages/com.unity.physics@1.3/manual/index.html`.

[33] Yiheng Xie, Towaki Takikawa, Shunsuke Saito, Or Litany, Shiqin Yan, Numair Khan, Federico Tombari, James Tompkin, Vincent Sitzmann, and Srinath Sridhar. Neural Fields in Visual Computing and Beyond. *Computer Graphics Forum*, 41(2):641–676, 2022. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.14505.

[34] Paul Zikas, Antonis Protopsaltis, Nick Lydatakis, Mike Kentros, Stratos Geronikolakis, Steve Kateros, Manos Kamarianakis, Giannis Evangelou, Achilleas Filippidis, Eleni Grigoriou, Dimitris Angelis, Michail Tamiolakis, Michael Dodis, George Kokiadis, John Petropoulos, Maria Pateraki, and George Papagiannakis. Mages 4.0: Accelerating the world's transition to VR training and democratizing the authoring of the Medical Metaverse. *IEEE Computer Graphics and Applications*, 43(2):43–56, 2023.